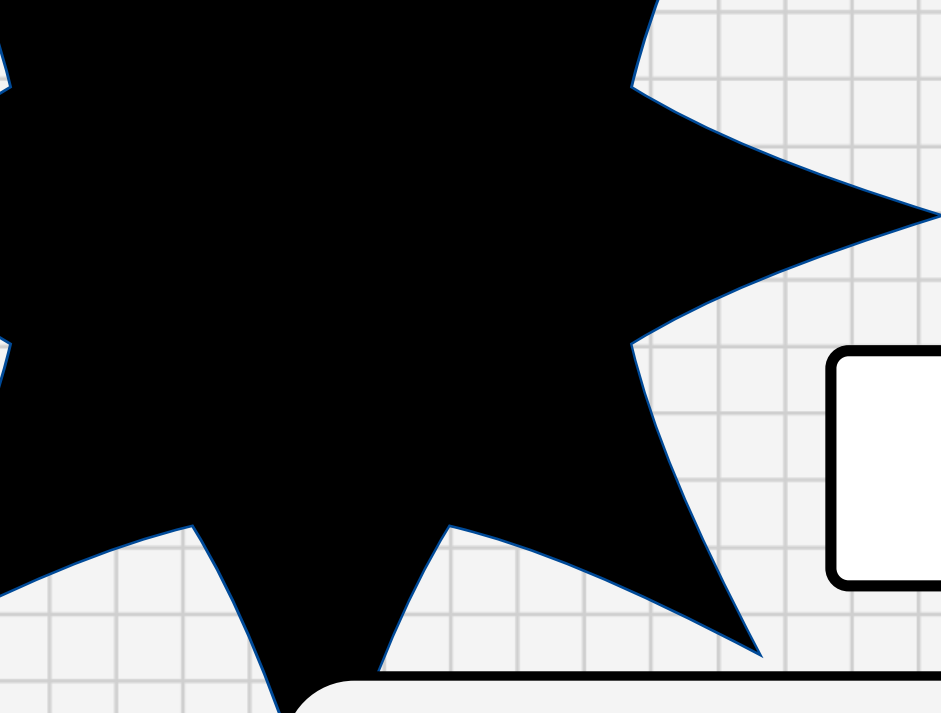
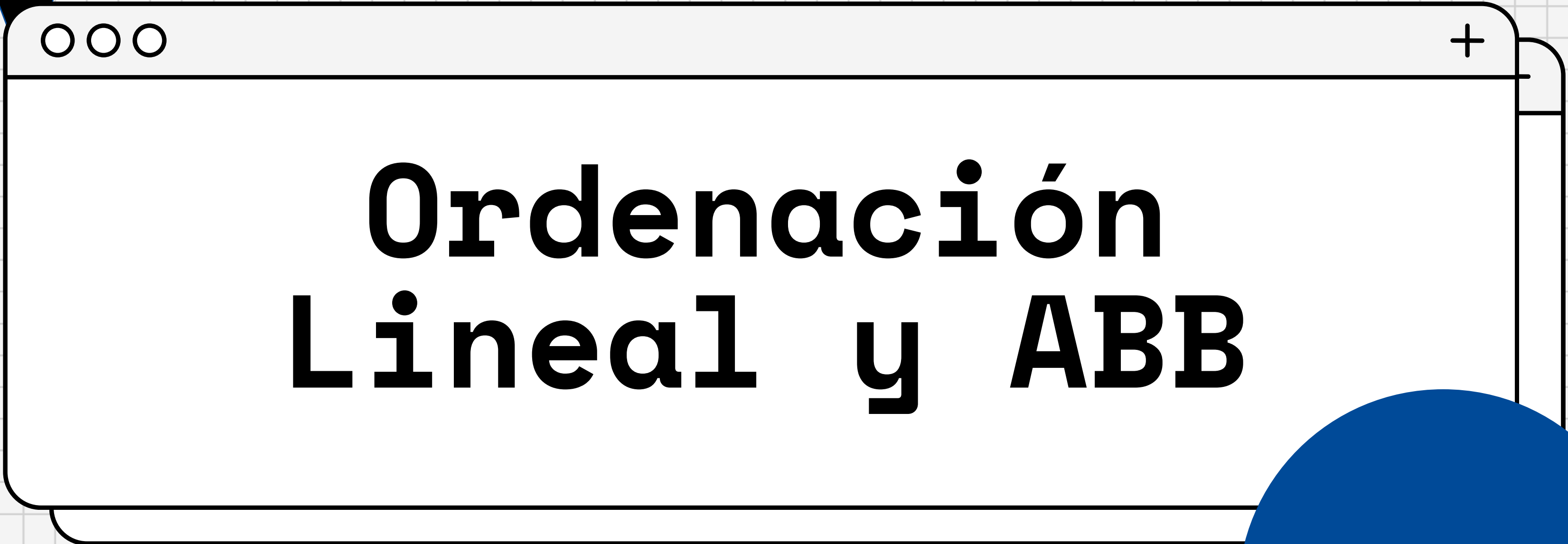
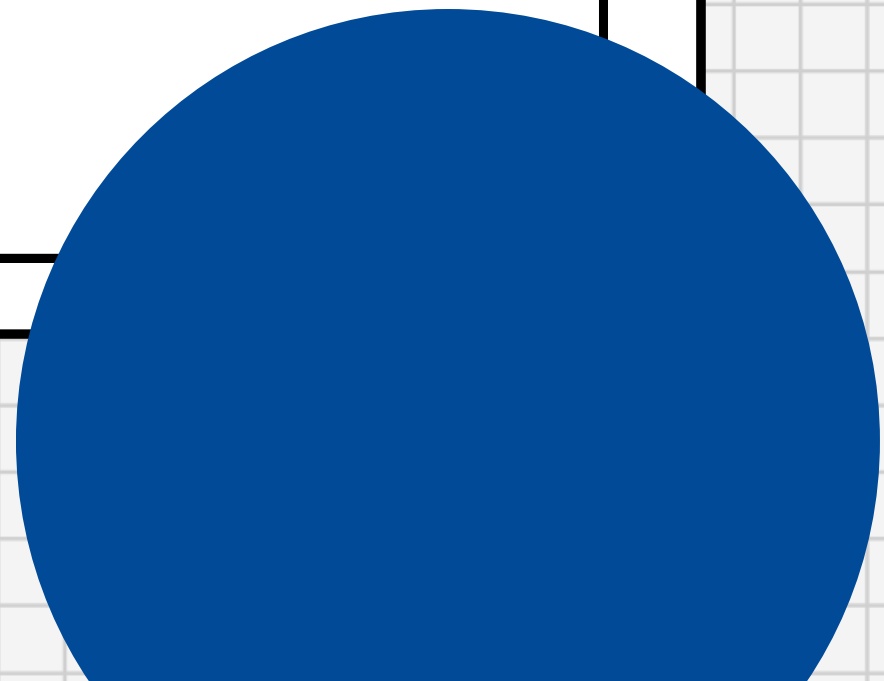




Ayudantía N° 7



# Ordenación Lineal y ABB



# Repaso

# Ordenación Lineal



# CountingSort

Input: Arreglo **A**, con cota superior entero natural **k**

Output: Secuencia ordenada **B**

Inplace o not inplace?? Por qué?

Complejidad:  $O(n+k)$

Si  $k$  es  $O(n) \rightarrow$  Complejidad:  $O(n)$

*Es mejor complejidad que  $O(n \log(n))$ !!*

*Ojo: C termina siendo cuántos elementos tengo antes más 1.*

CountingSort ( $A, k$ ):

```
1   $B[0 \dots n-1] \leftarrow$  arreglo vacío de  $n$  celdas
2   $C[0 \dots k] \leftarrow$  arreglo vacío de  $k+1$  celdas
3  for  $i = 0 \dots k$  :
4       $C[i] \leftarrow 0$ 
5  for  $j = 0 \dots n-1$  :
6       $C[A[j]] \leftarrow C[A[j]] + 1$ 
7  for  $p = 1 \dots k$  :
8       $C[p] \leftarrow C[p] + C[p-1]$ 
9  for  $r = n-1 \dots 0$  :
10      $B[C[A[r]] - 1] \leftarrow A[r]$ 
11      $C[A[r]] \leftarrow C[A[r]] - 1$ 
12  return  $B$ 
```

# CountingSort

**A** =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

Identificamos  $n = 6$  y  $k = 9$ . El llamado a la función será:  
`CountingSort(A, 9)`

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**B =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
|   |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   |   |   |   |   |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

Se definen B y C como arreglos vacíos de largo n y k+1 respectivamente

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**B =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
|   |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

**Se inicializan todos los elementos de C como 0**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

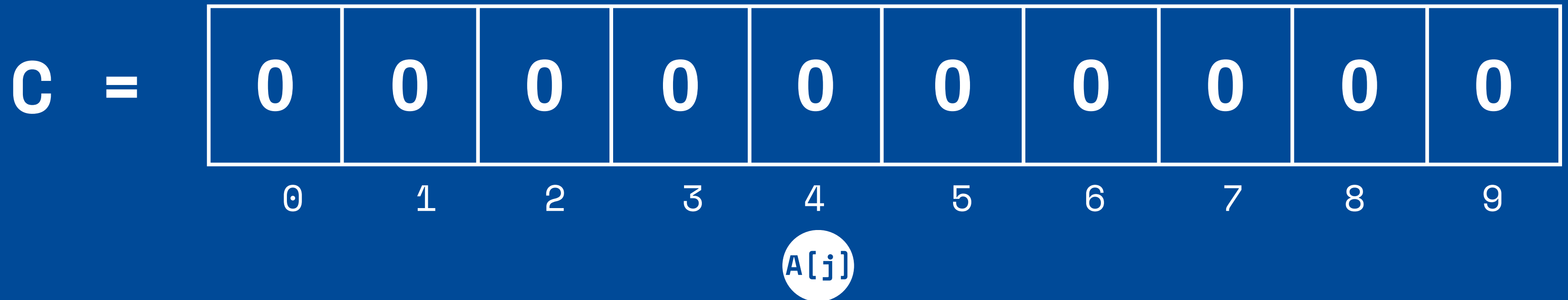
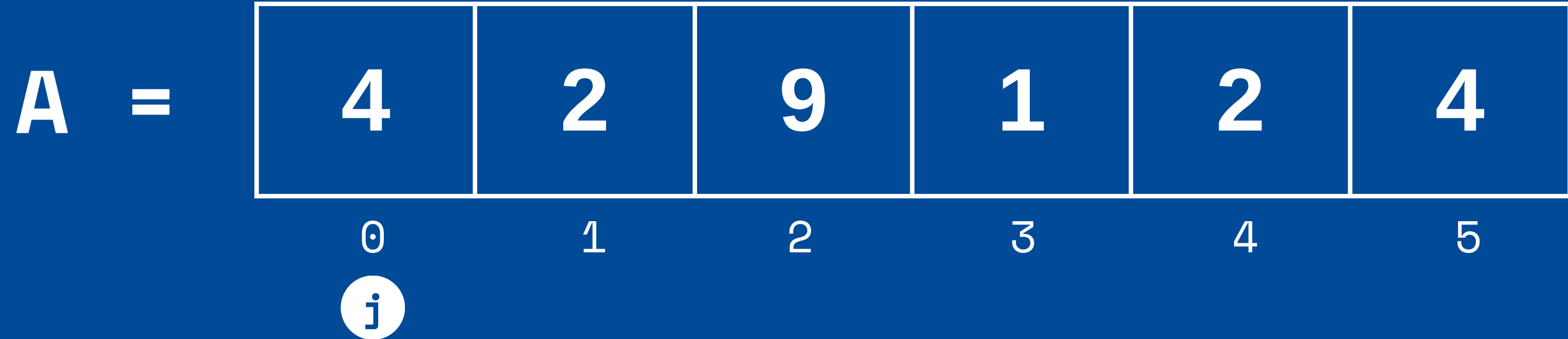
**B =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
|   |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 |

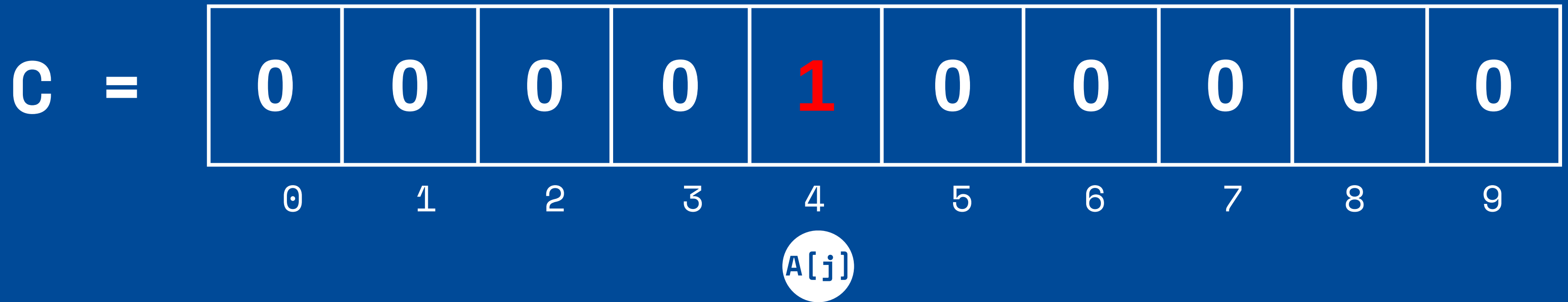
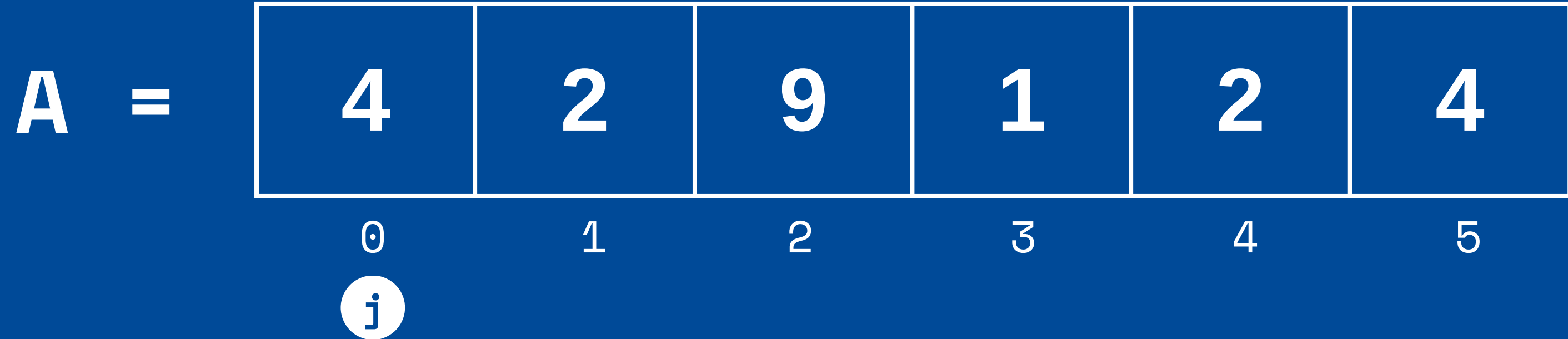
**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

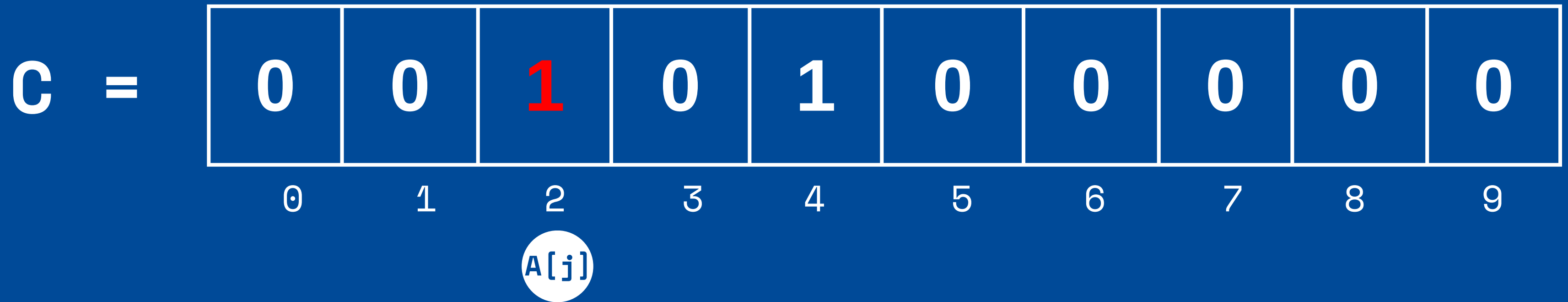
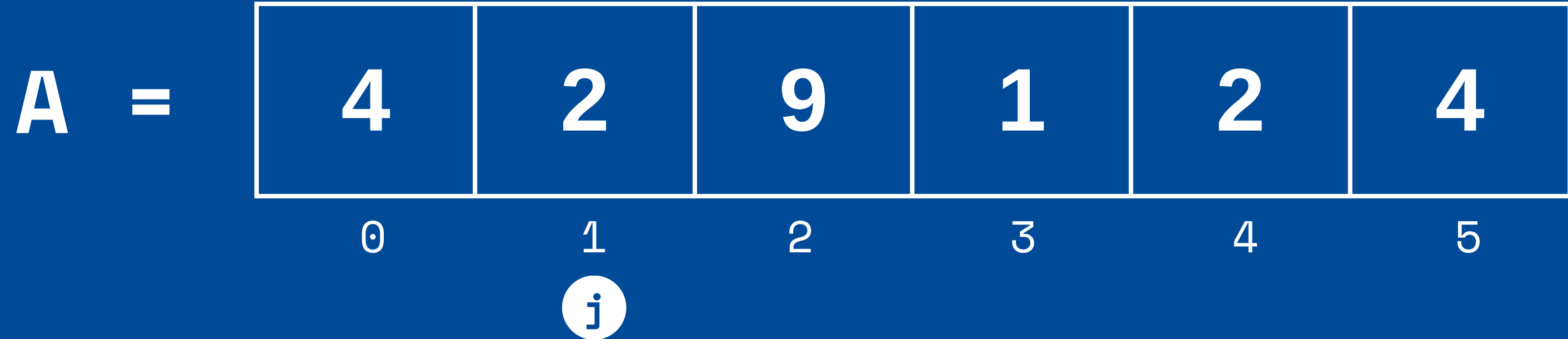
Utilizamos **j** para ir recorriendo **A** y actualizando **C**



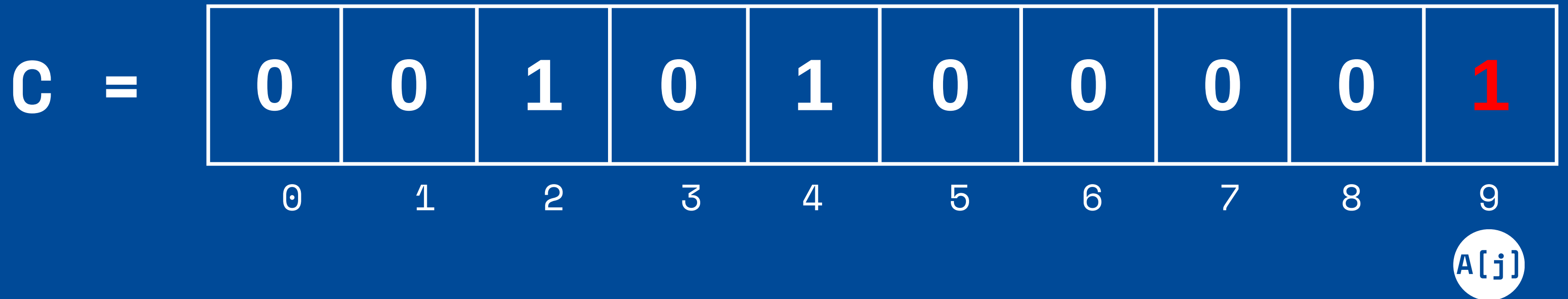
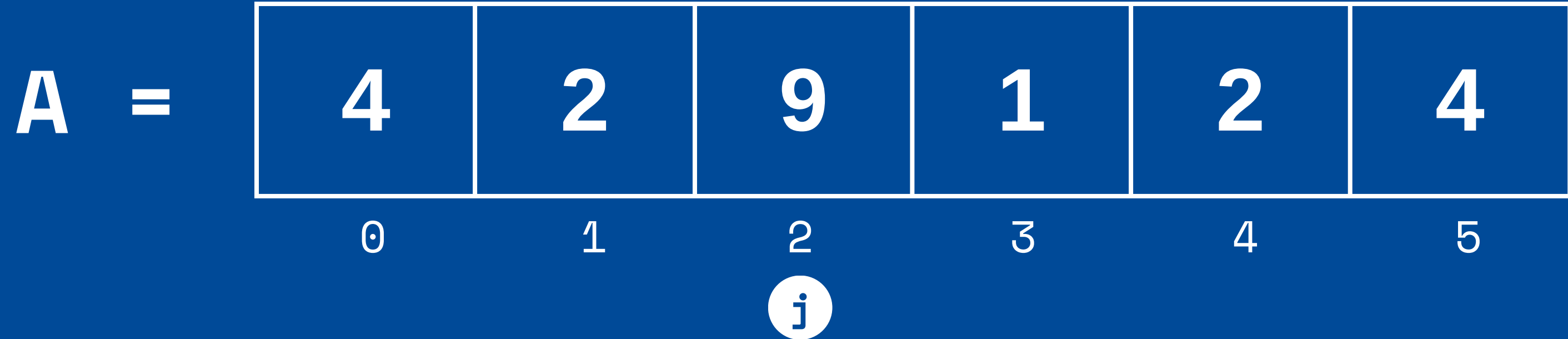
Utilizamos **j** para ir recorriendo **A** y actualizando **C**  
**j** inicia siendo 0



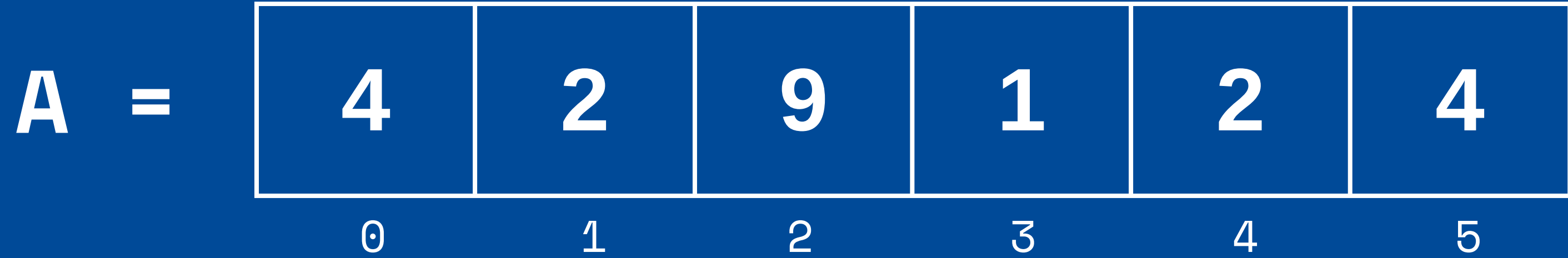
Sumamos 1 a C[A[j]]



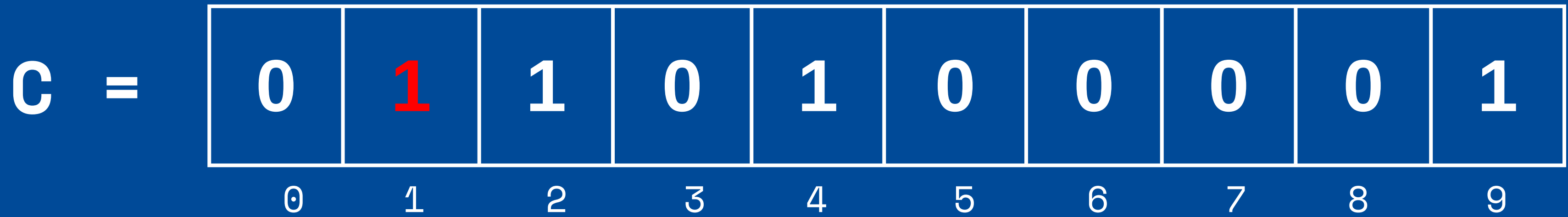
Actualizamos **j**, y sumamos 1 a **C[A[j]]**



Actualizamos **j**, y sumamos 1 a **C[A[j]]**

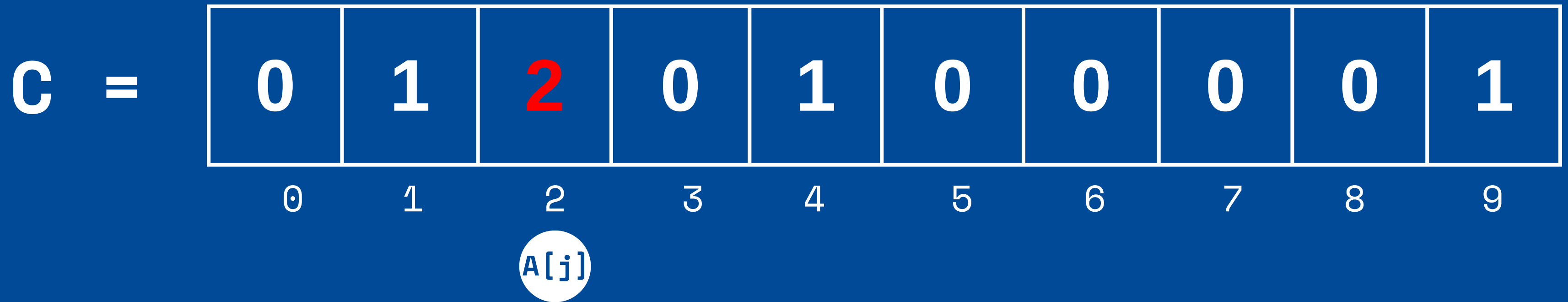
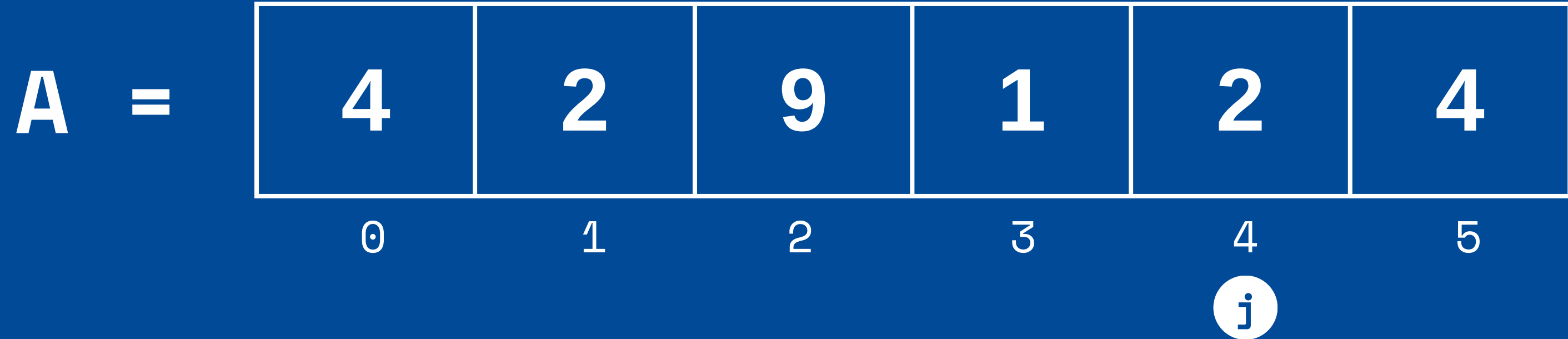


**j**

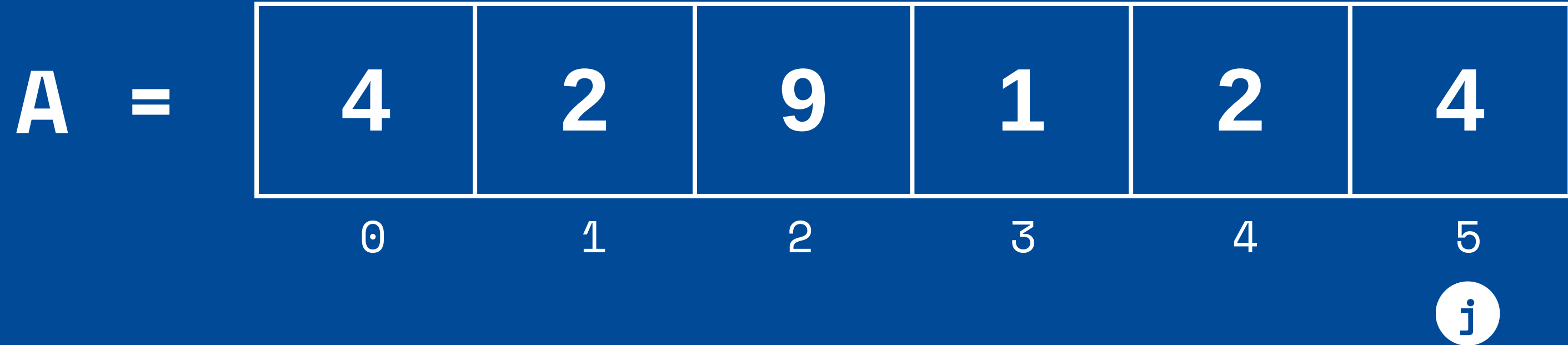


**A[j]**

Actualizamos **j**, y sumamos 1 a **C[A[j]]**



Actualizamos **j**, y sumamos 1 a **C[A[j]]**



Actualizamos **j**, y sumamos 1 a **C[A[j]]**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 0 | 2 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

Recorreremos y actualizamos C haciendo uso de  
 $p = 1 \dots k$

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 0 | 2 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

**p**

Actualizamos  $C[p]$  sumándole el valor del elemento anterior

$$C[p] = C[p] + C[p-1]$$

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 0 | 2 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

**p**

Actualizamos  $C[p]$  sumándole el valor del elemento anterior

$$C[p] = C[p] + C[p-1]$$

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 2 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

p

Actualizamos  $C[p]$  sumándole el valor del elemento anterior

$$C[p] = C[p] + C[p-1]$$

**A** =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C** =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 1 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

p

Fastforward...

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

p

Última iteración de p

**A** =

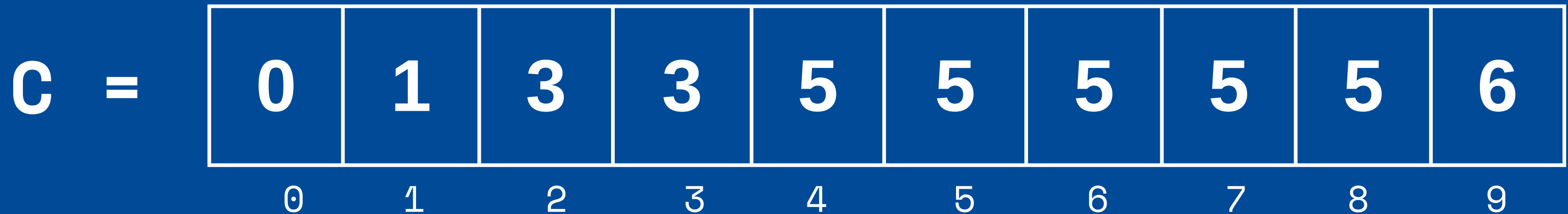
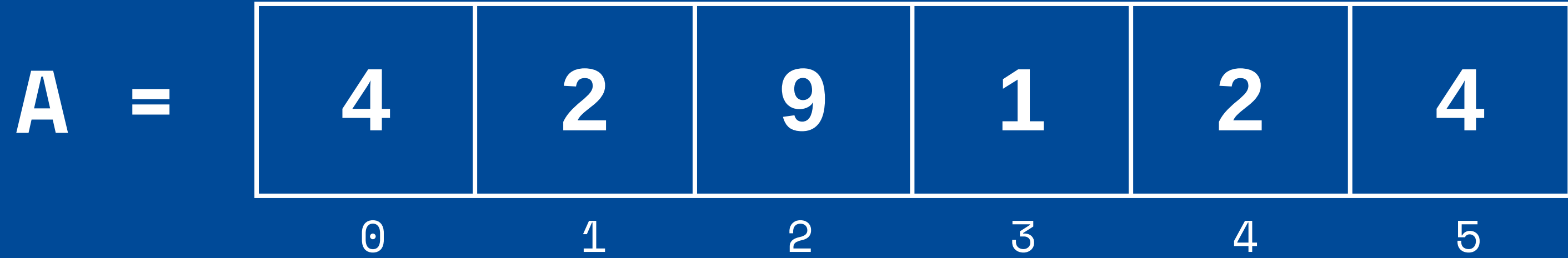
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C** =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

p

Ojo,  $C[k]$  coincide exactamente con  $n$



En este punto  $C[x]$  representa cuanto elementos menores o iguales a  $x$   
hay en el arreglo A

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 6 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

Ahora empezamos a poblar B, para esto recorreremos  
A con  $r = n-1 \dots 0$   
C con  $A[r]$   
y B con  $C[A[r]] - 1 (*)$

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|  |  |  |  |   |  |
|--|--|--|--|---|--|
|  |  |  |  | 4 |  |
|--|--|--|--|---|--|

0

1

2

3

4

5

\*

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

A =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

r

B =

|  |  |  |  |   |  |
|--|--|--|--|---|--|
|  |  |  |  | 4 |  |
|--|--|--|--|---|--|

0

1

2

3

4

5

\*

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

C =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

A[r]

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|  |  |  |  |   |  |
|--|--|--|--|---|--|
|  |  |  |  | 4 |  |
|--|--|--|--|---|--|

0

1

2

3

4

5

\*

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|  |  |   |  |   |  |
|--|--|---|--|---|--|
|  |  | 2 |  | 4 |  |
|--|--|---|--|---|--|

0

1

2

3

4

5

\*

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|  |  |   |  |   |  |
|--|--|---|--|---|--|
|  |  | 2 |  | 4 |  |
|--|--|---|--|---|--|

0

1

2

3

4

5

**\***

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

A =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

r

B =

|   |  |   |  |   |  |
|---|--|---|--|---|--|
| 1 |  | 2 |  | 4 |  |
|---|--|---|--|---|--|

0

1

2

3

4

5

\*

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

C =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

A[r]

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|   |  |   |  |   |  |
|---|--|---|--|---|--|
| 1 |  | 2 |  | 4 |  |
|---|--|---|--|---|--|

0

1

2

3

4

5

\*

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 |   |

**r**

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

**B =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 |   | 2 |   | 4 | 9 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**\***

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 5 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

**A[r]**

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|   |  |   |  |   |   |
|---|--|---|--|---|---|
| 1 |  | 2 |  | 4 | 9 |
|---|--|---|--|---|---|

0

1

2

3

4

5

**\***

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 2 | 3 | 4 | 5 | 5 | 5 | 5 | 5 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

A =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

r

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

B =

|   |   |   |  |   |   |
|---|---|---|--|---|---|
| 1 | 2 | 2 |  | 4 | 9 |
|---|---|---|--|---|---|

0

1

2

3

4

5

\*

C =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 3 | 4 | 5 | 5 | 5 | 5 | 5 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

A[r]

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

5

**r**

**B =**

|   |   |   |  |   |   |
|---|---|---|--|---|---|
| 1 | 2 | 2 |  | 4 | 9 |
|---|---|---|--|---|---|

0

1

2

3

4

5

\*

**C =**

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 3 | 4 | 5 | 5 | 5 | 5 | 5 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

**A[r]**

A =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
|---|---|---|---|---|---|

0

1

2

3

4

r

Actualizamos B  
con el valor  
que irá en ese  
espacio, y  
restamos 1 en C

B =

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 2 | 4 | 4 | 9 |
|---|---|---|---|---|---|

0

1

2

3

4

5

\*

C =

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 3 | 3 | 5 | 5 | 5 | 5 | 5 |
|---|---|---|---|---|---|---|---|---|---|

0

1

2

3

4

5

6

7

8

9

A[r]

**A =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 2 | 9 | 1 | 2 | 4 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**B =**

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 2 | 4 | 4 | 9 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**Finalmente obtenemos B, arreglo ordenado  
con los mismos elementos de A**



# RadixSort



Si cada dígito puede tomar  $k$  valores distintos, entonces la complejidad es  $O(d(n+k))$

Si  $d$  es constante y  $k$  es  $O(n)$ , entonces la complejidad es  $O(n)$

RadixSort( $A, d$ ):

**for**  $j = 0 \dots d - 1$  :

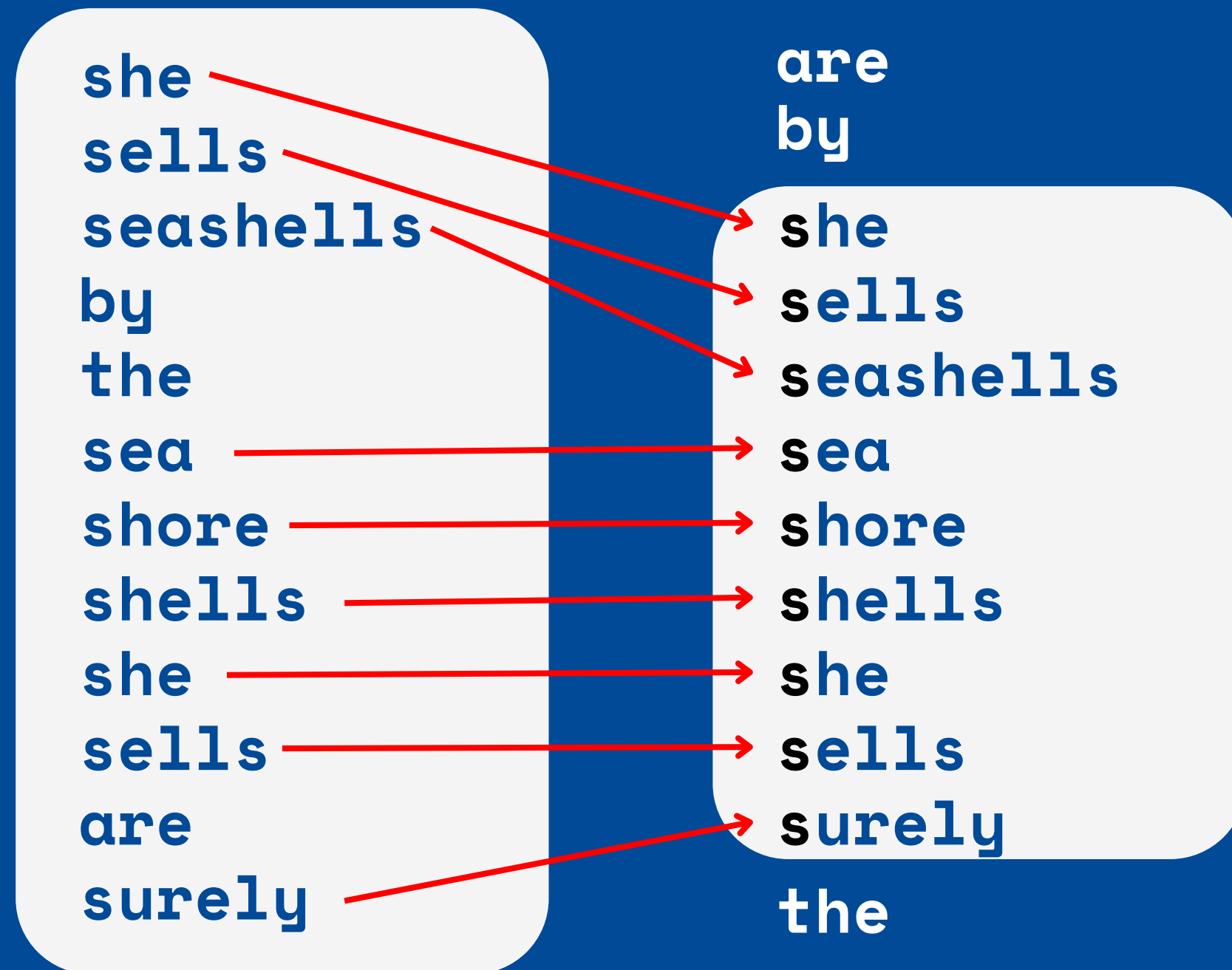
    StableSort( $A, j$ )

      ▷ algoritmo de ordenación estable por  $j$ -ésimo dígito menos significativo

# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

# RadixSort



# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

are  
by

she  
sells  
seashells  
sea  
shore  
shells  
she  
sells  
surely

the

# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

are  
by

she  
sells  
seashells  
sea  
shore  
shells  
she  
sells  
surely

the

are  
by

sells  
seashells  
sea  
sells

she  
shore  
shells  
she  
surely  
the

# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

are  
by  
she  
sells  
seashells  
sea  
shore  
shells  
she  
sells  
surely  
the

are  
by  
sells  
seashells  
sea  
sells  
she  
shore  
shells  
she  
surely  
the

# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

are  
by  
she  
sells  
seashells  
sea  
shore  
shells  
she  
sells  
surely  
the

are  
by  
sells  
seashells  
sea  
sells  
she  
shore  
shells  
she  
surely  
the

are  
by  
seashells  
sea  
sells  
sells  
she  
shore  
shells  
she  
surely  
the

# RadixSort

she  
sells  
seashells  
by  
the  
sea  
shore  
shells  
she  
sells  
are  
surely

are  
by  
she  
sells  
seashells  
sea  
shore  
shells  
she  
sells  
surely  
the

are  
by  
sells  
seashells  
sea  
sells  
she  
shore  
shells  
she  
surely  
the

are  
by  
seashells  
sea  
sells  
sells  
she  
shore  
shells  
she  
surely  
the

# Árboles Binarios de Búsqueda (ABB)



# ¿Qué es un árbol binario?



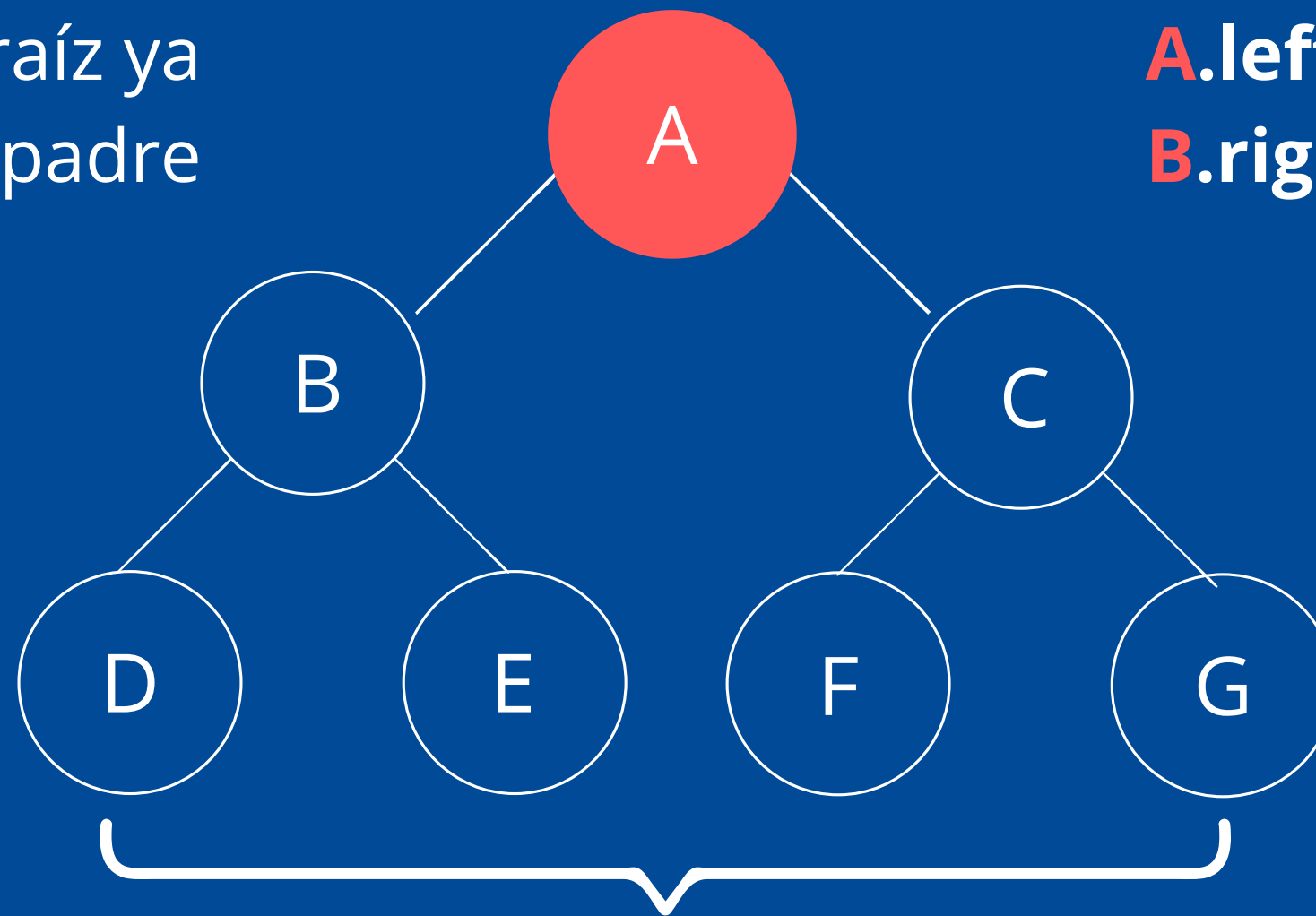
Un AB cumple con que cada nodo  $x$ :

- Si tiene padre, este sea  $x.p$
- Si no tiene padre,  $x$  es la raíz
- Tiene dos punteros a nodos hijos izquierdo ( $x.left$ ) y derecho ( $x.right$ ), si existen
- Si  $x$  no tiene “hijos”, entonces  $x$  es una hoja

# Ejemplo de AB

**A** es el nodo raíz ya que no posee padre

**A.left** apunta a B  
**B.right** apunta a C



**D, E, F, G** son hojas (no poseen hijos)



## ¿Qué es un ABB?

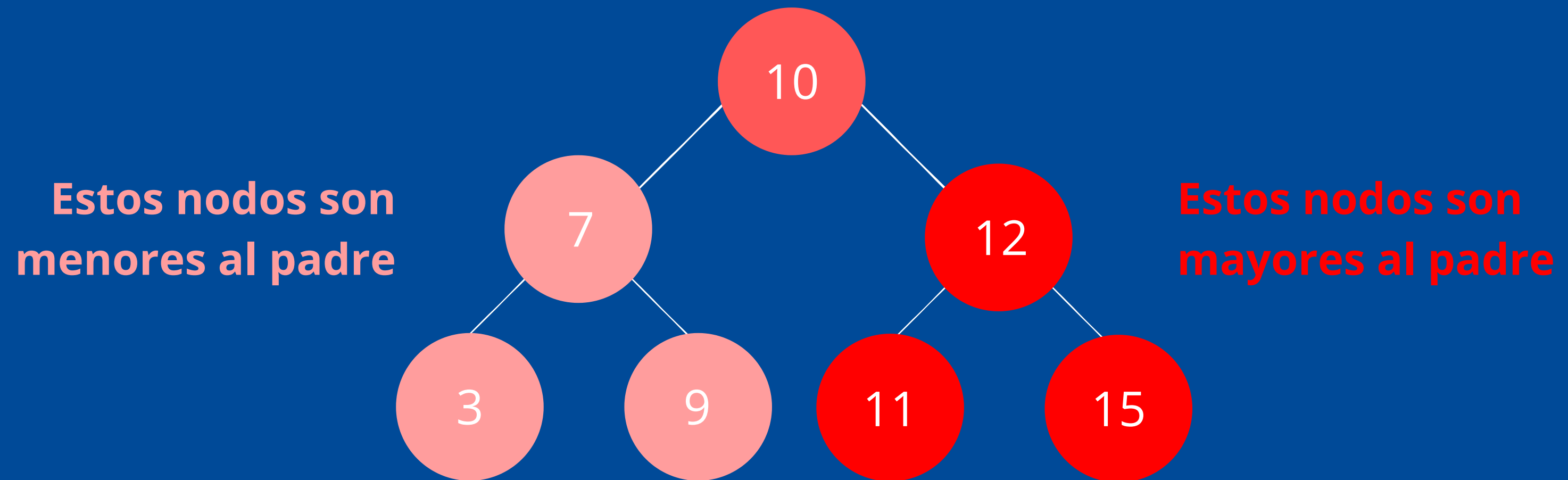


Una **árbol binario de búsqueda (ABB)** es una estructura de datos que es un tipo de AB. Esta almacena tuplas (llave, valor) con punteros a sus hojas.

Un ABB, al igual que los AB, se divide en dos ramas, un **hijo izquierdo** y uno **derecho**, que a su vez también son ABBs.

La propiedad más importante de los ABBs es que se cumple que la **llave** almacenada en el **hijo izquierdo** es **menor al padre**, mientras que la del **hijo derecho** será **mayor**.

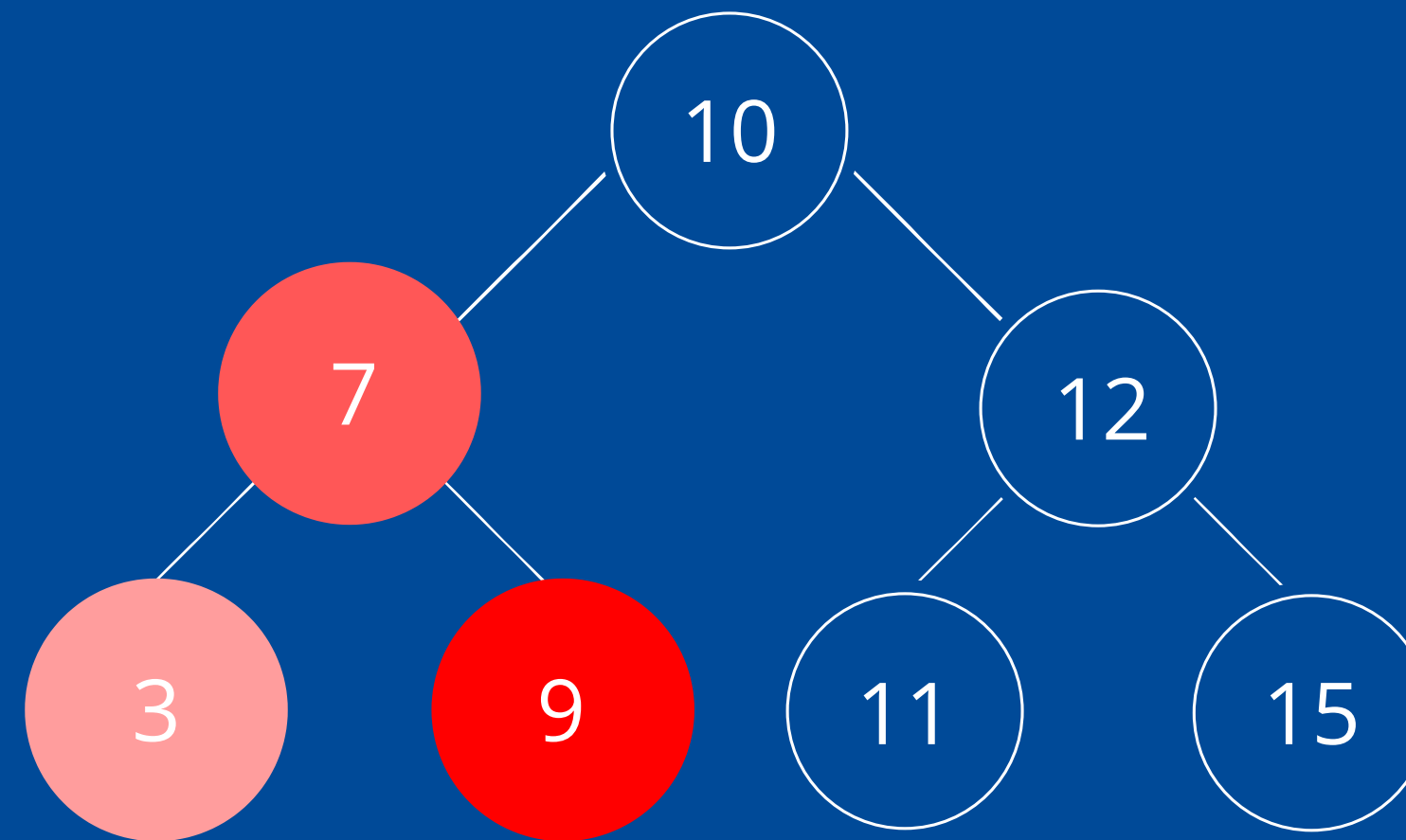
# Ejemplo de ABB



Ahora cumplimos propiedades de ABB!

# Ejemplo de ABB

Es importante recordar que los hijos también serán ABBs



Este nodo es  
menor al padre

Este nodo es  
mayor al padre

## Operaciones de ABB



```
Search(A, p, k):  
1  if  $A = \emptyset \vee A.key = k$  :  
2      return (A, p)  
3  if  $k < A.key$  :  
4      return Search(A.left, A, k)  
5  return Search(A.right, A, k)
```

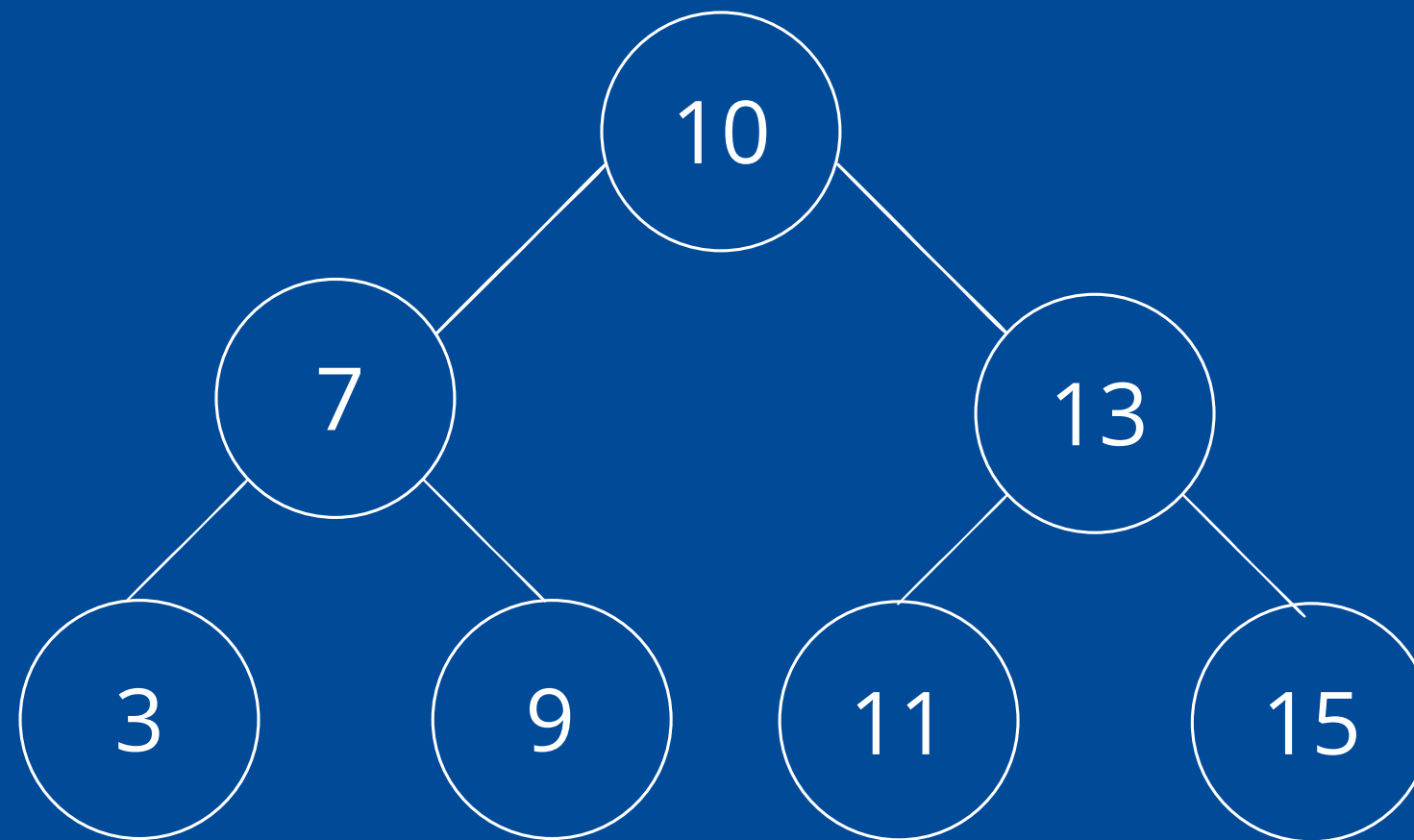
Input: un ABB **A**, su ABB padre **p** y la llave que buscamos **k**

Output: un **ABB**, cuya raíz tiene como llave **k** en caso de que haya sido encontrada, o está vacío, y su **ABB** padre **p**

# Search en ABB

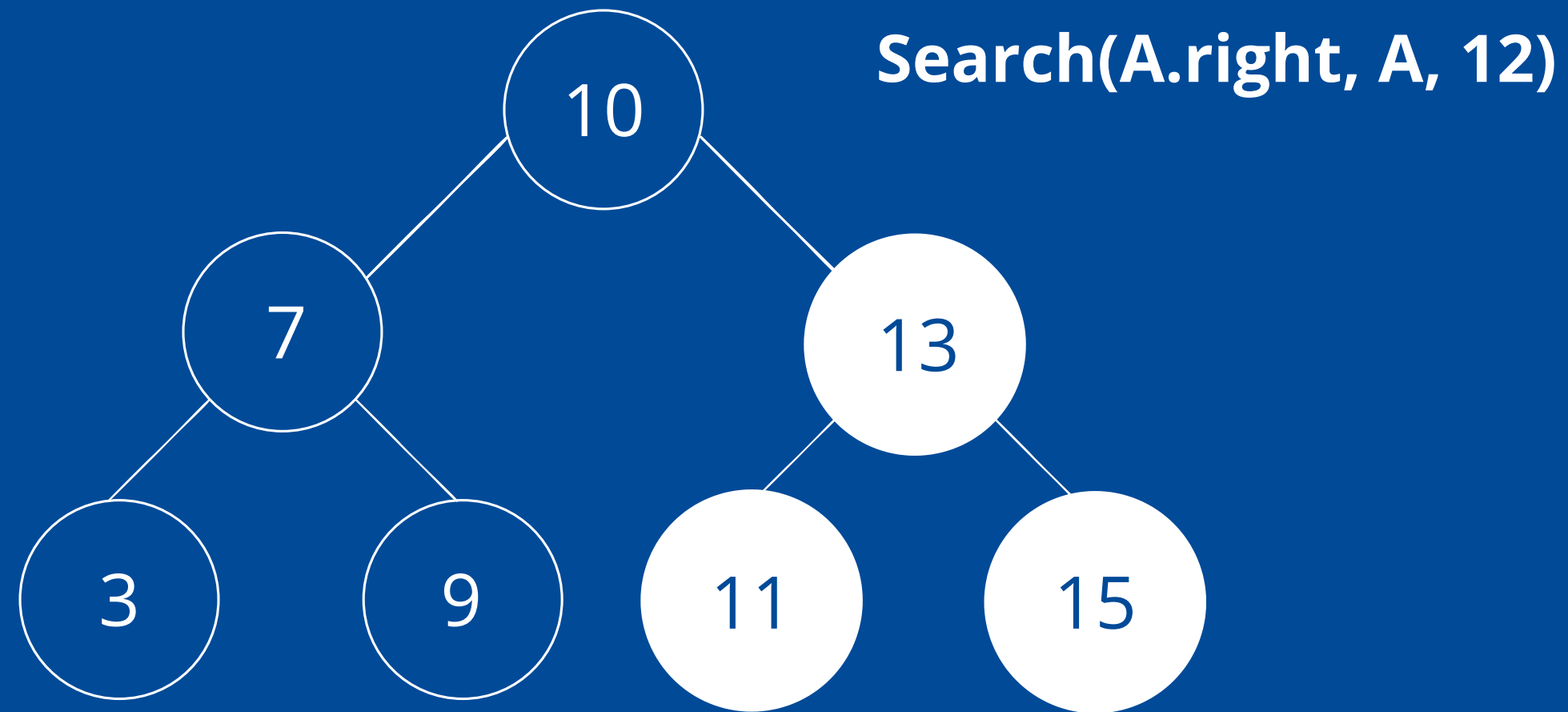
Digamos que deseo buscar un nodo con key 12

$\text{Search}(A, \emptyset, 12)$



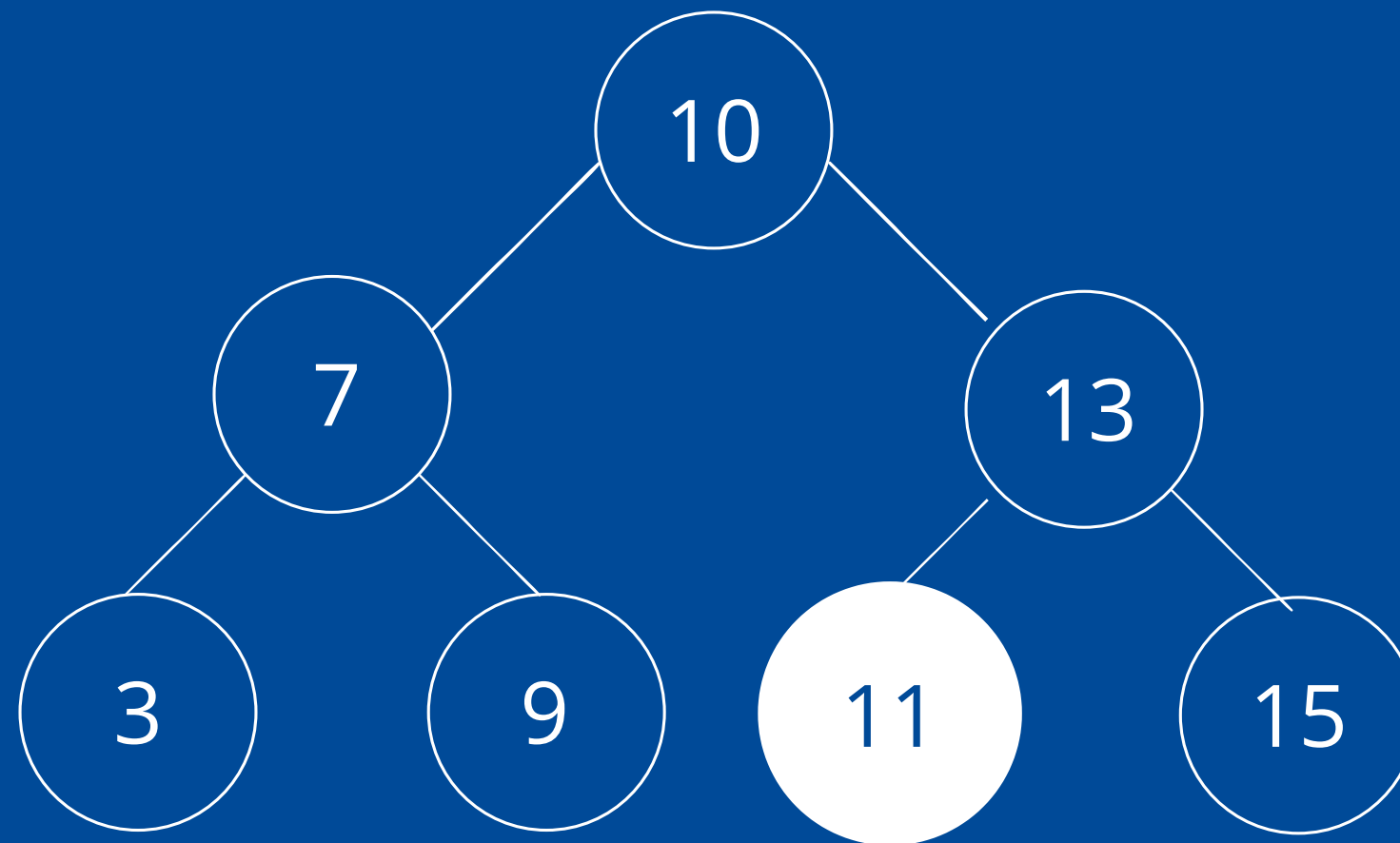
# Search en ABB

Dado que 12 es mayor a 10, buscamos en la rama derecha



# Search en ABB

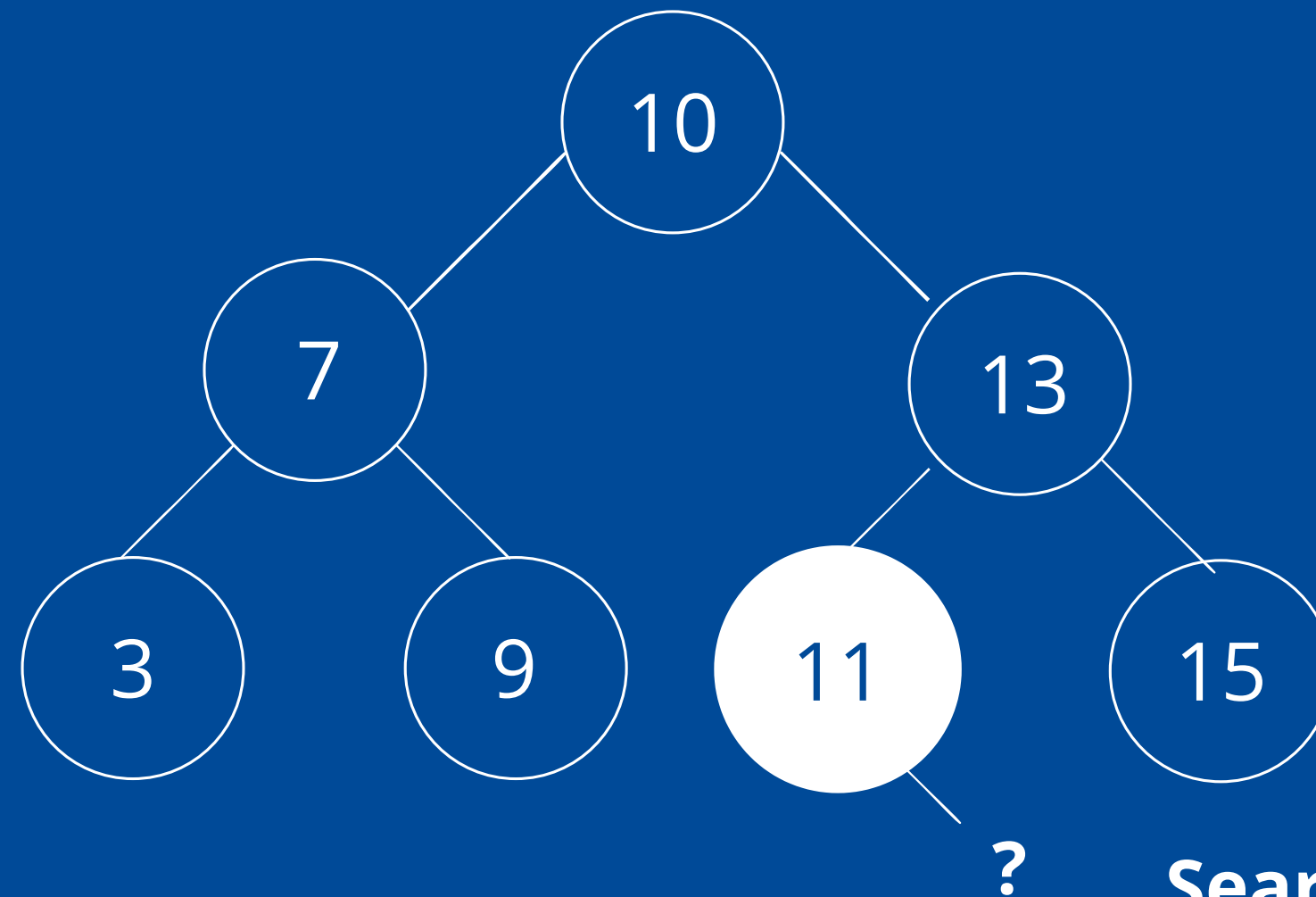
Dado que 12 es menor a 13, buscamos en la rama izquierda



**Search(A.right.left, A.right, 12)**

# Search en ABB

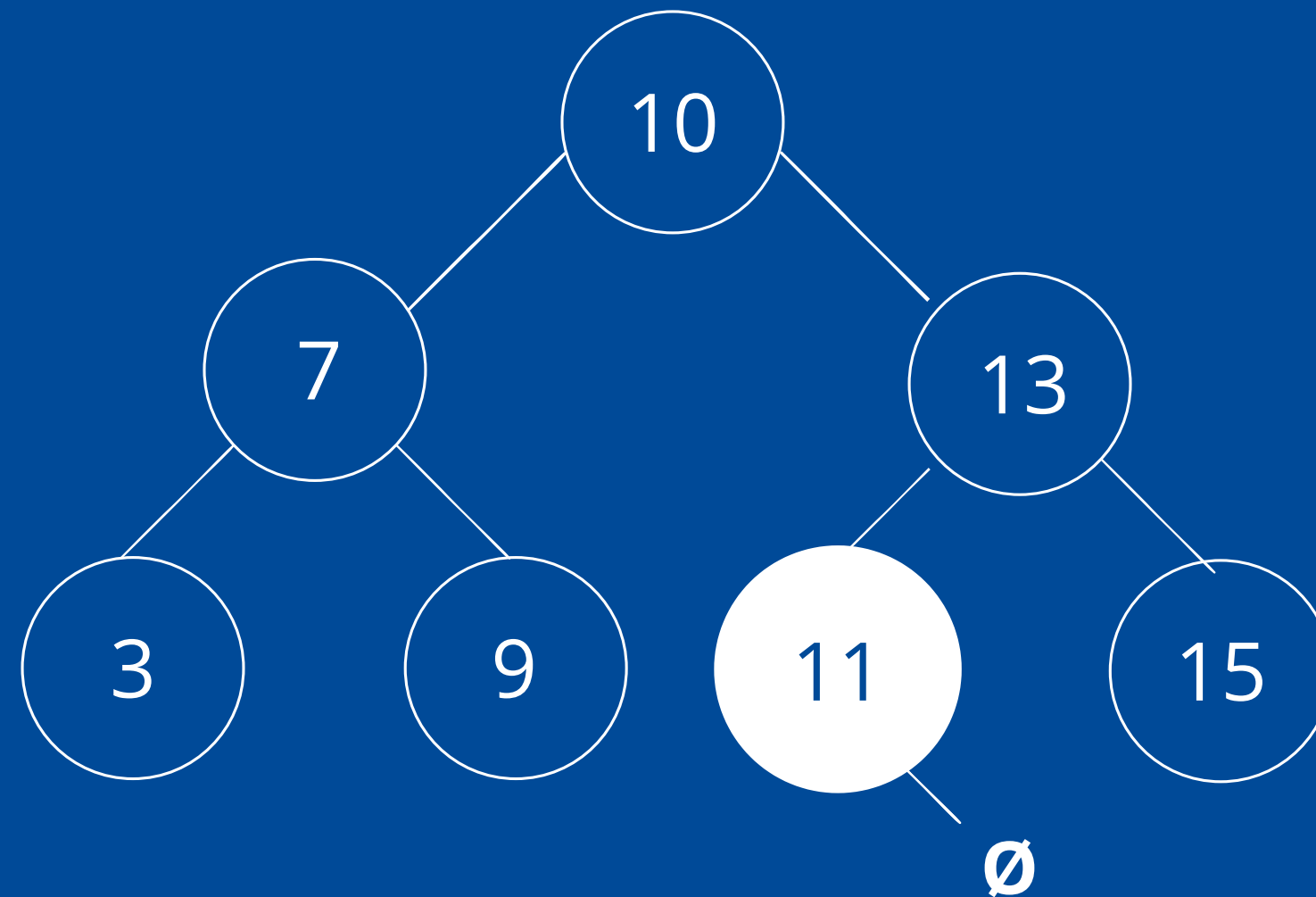
Como 11 no tiene hijos,  $A.\text{right.left.right} = \emptyset$



**Search( $A.\text{right.left.right}$ ,  
 $A.\text{right.left}$ , 12)**

# Search en ABB

Por lo tanto, retornamos (A.right.left.right, A.right.left)



# Operaciones de ABB



Insert ( $A, k, v$ ):

1     $(B, p) \leftarrow \text{Search}(A, \emptyset, k)$     ▷ versión que indica el padre

2    **if**  $B = \emptyset$  :

3         $B \leftarrow$  nodo vacío

4         $B.\text{key} \leftarrow k$

5        Conectar  $B$  al padre  $p$  en la posición adecuada

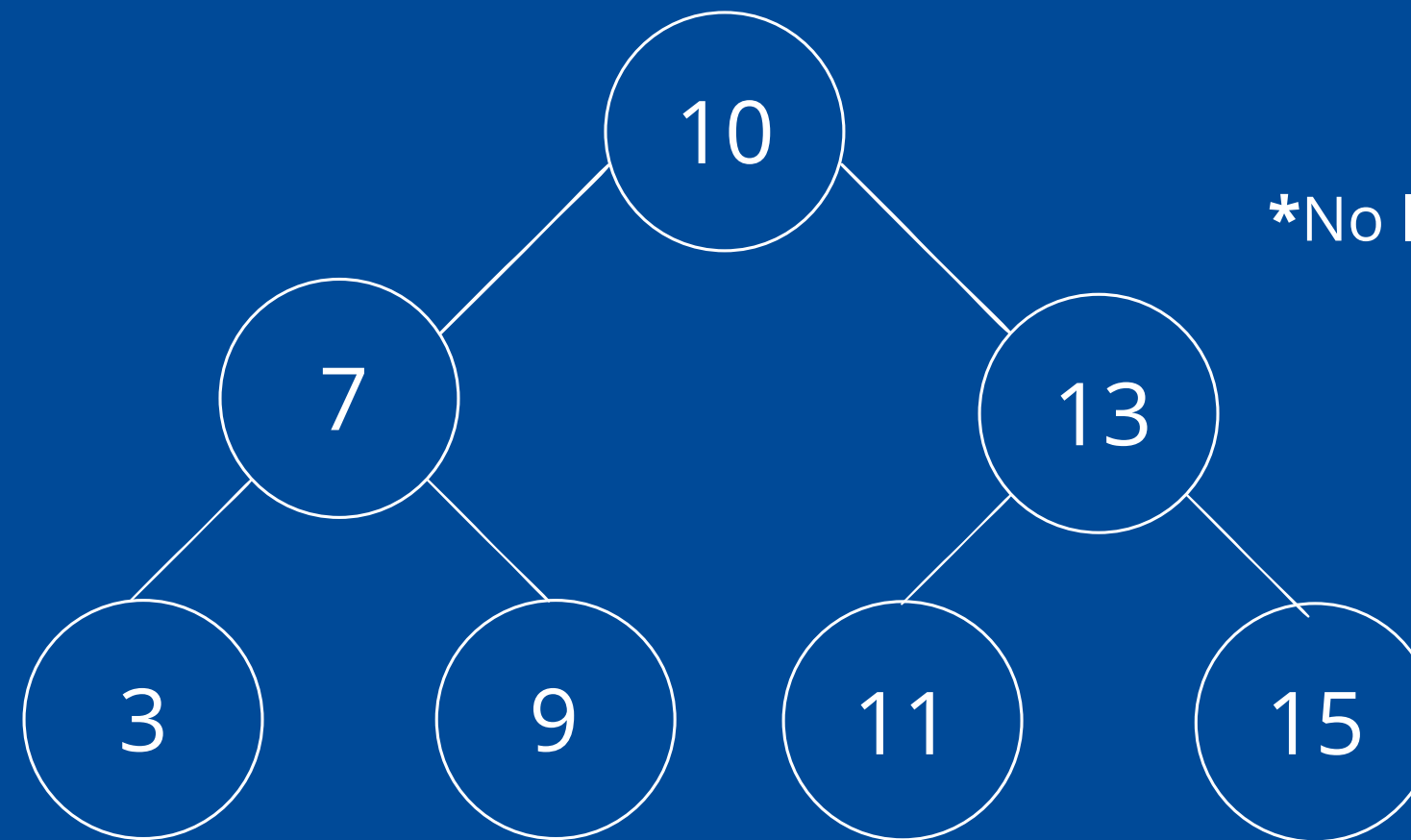
6         $B.\text{value} \leftarrow v$

Recibimos de input un ABB  
A, la llave que queremos  
insertar  $k$  y su valor  $v$ .

# Insert en ABB

Digamos que deseo buscar un nodo con key 12

Insert(A, 12, v)

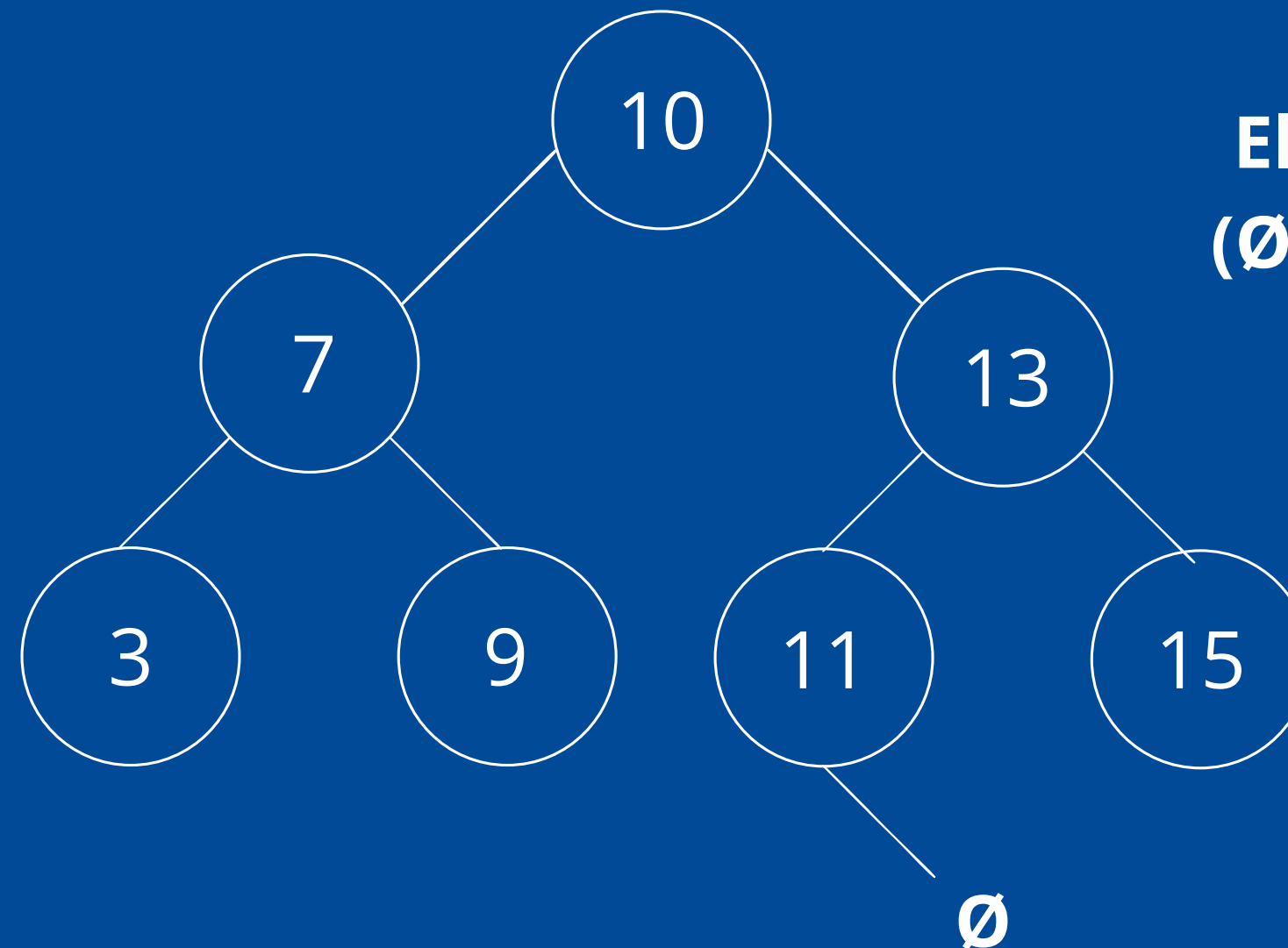


\*No le daremos relevancia al valor  
por ahora

# Insert en ABB

Digamos que deseo buscar un nodo con key 12

Insert(A, 12, v)



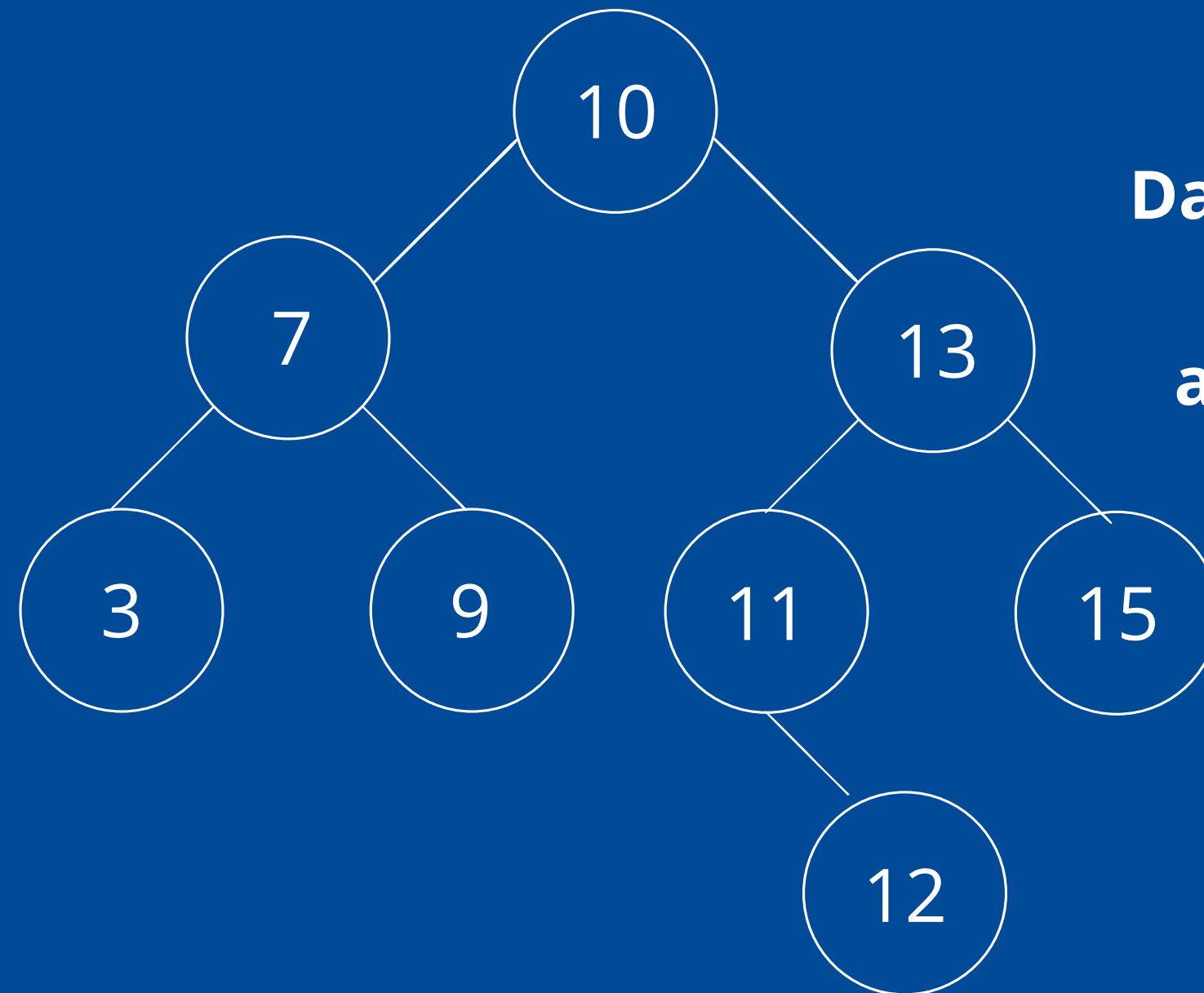
El Search de 12 nos retornó  
( $\emptyset$ , A.right.left), por lo tanto

$B = \emptyset$

$p = A.\text{right.left}$

# Insert en ABB

**Insert(A, 12, v)**

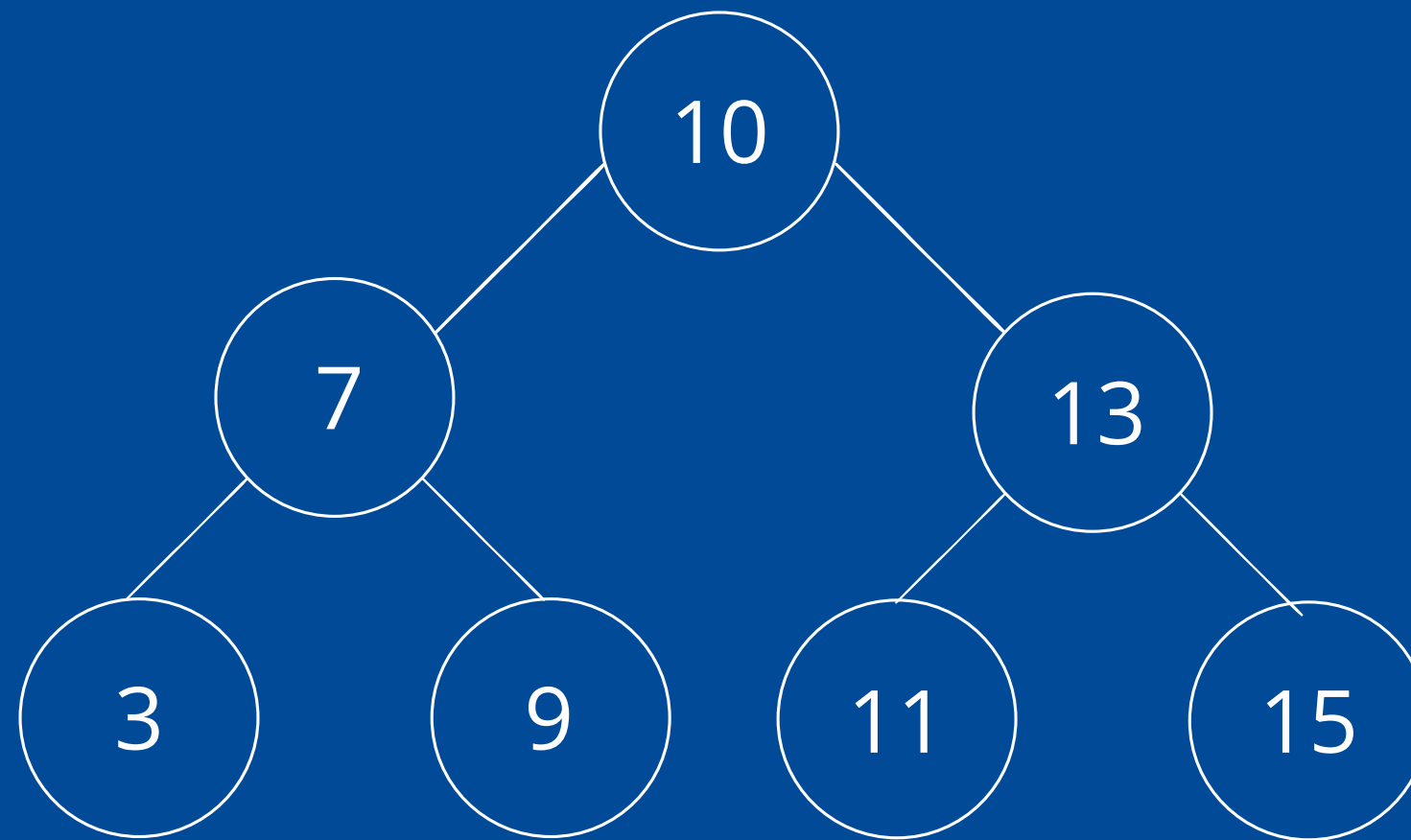


**Dado eso, creamos un  
nodo nuevo b, le  
asignamos key 12 y  
value v**

# Insert en ABB

Si quisieramos insertar un valor con key ya existente, por convención del código, solamente se actualiza v

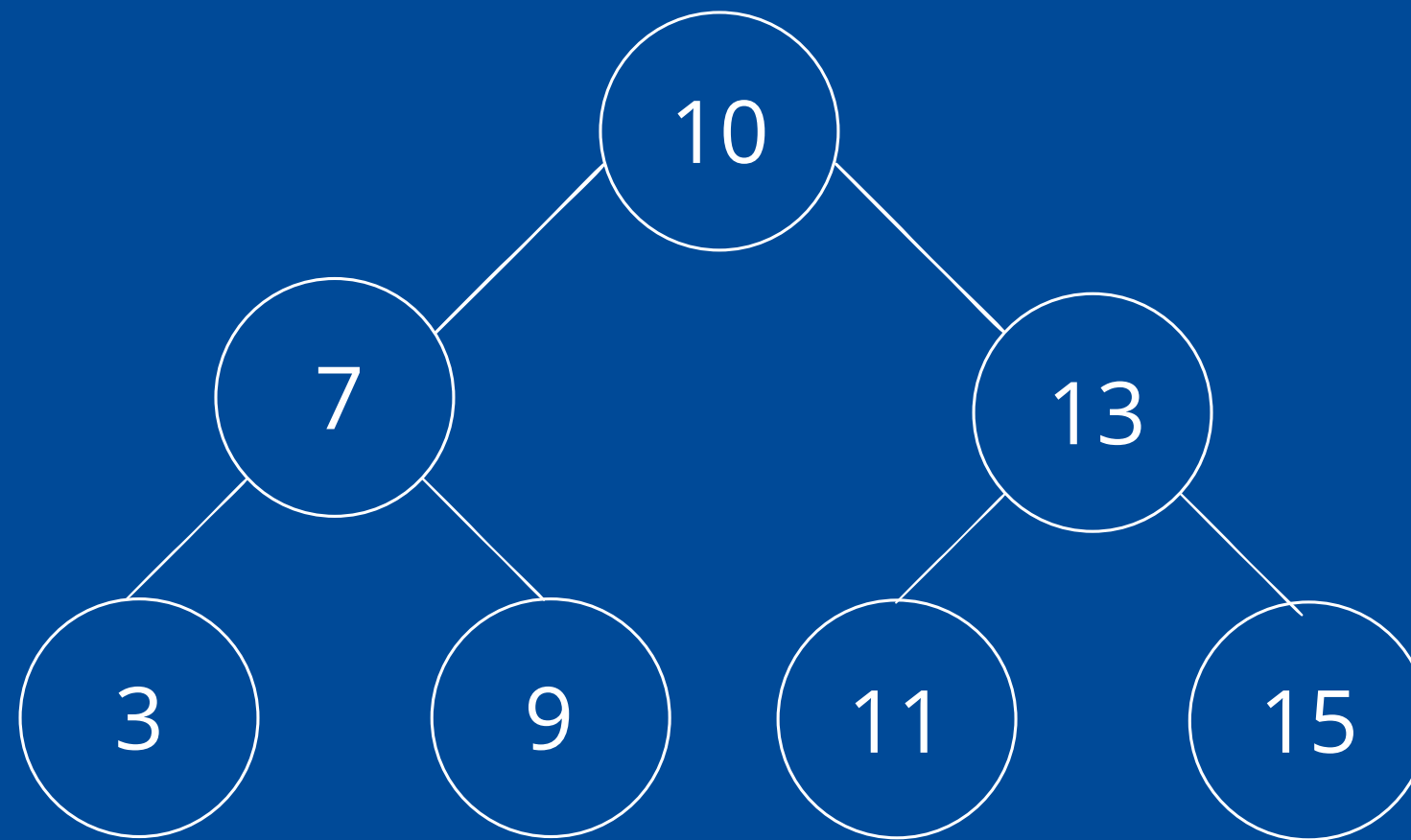
ej: Insert(A, 13, v)



# Insert en ABB

Esto podría verse más como una operación Update que Insert, pero el Insert debe ser capaz de manejar el caso

ej: Insert(A, 13, v)



# Operaciones de ABB



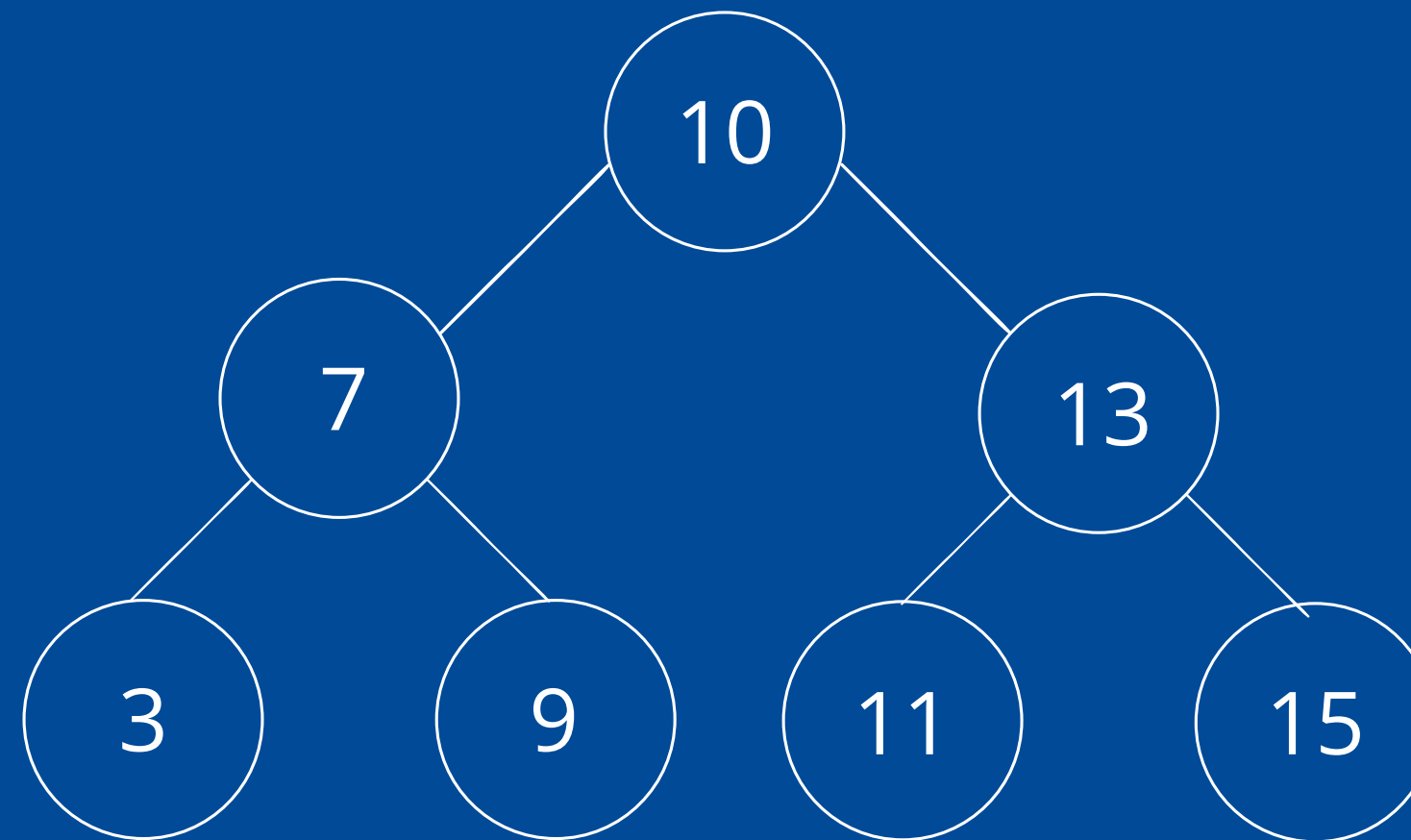
```
Delete (A, k):  
1  (D, p) ← Search(A, ∅, k)  ▷ Permite saber el padre de D  
2  if D ≠ ∅ :  
3      if D es hoja : D ← ∅ y se elimina la referencia en p  
4      elif D tiene un solo hijo H : D ← H y se actualiza p  
5      else:  
6          R ← Min(D.right)  
7          t ← R.right  
8          D.key ← R.key  
9          D.value ← R.value  
10         R ← t
```

Recibimos de input un ABB  
A, la llave que queremos  
eliminar k.

# Delete en ABB

Digamos que deseo eliminar el nodo con key 13

Delete(A, 13)



Search(A,  $\emptyset$ , 13)  
retornará (A.right, A)  
Por lo tanto  
D=A.right,  
p=A

# Delete en ABB

Digamos que deseo eliminar el nodo con key 13

Delete(A, 13)

else:

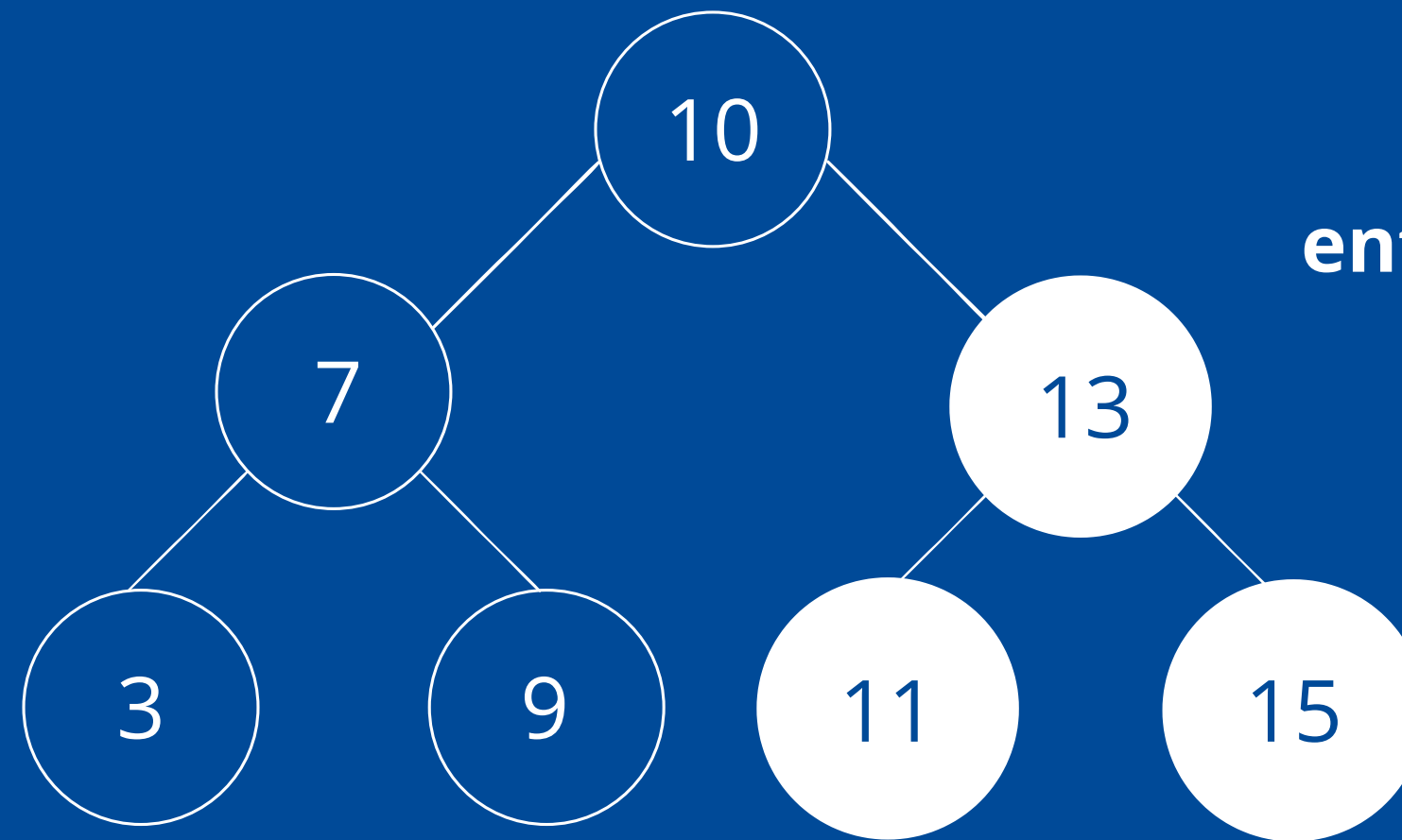
$R \leftarrow \text{Min}(D.\text{right})$

$t \leftarrow R.\text{right}$

$D.\text{key} \leftarrow R.\text{key}$

$D.\text{value} \leftarrow R.\text{value}$

$R \leftarrow t$



Como D posee 2 hijos,  
entramos al else del código

\*Diremos que este  
subarbol es D

# Delete en ABB

Digamos que deseo eliminar el nodo con key 13

Delete(A, 13)

else:

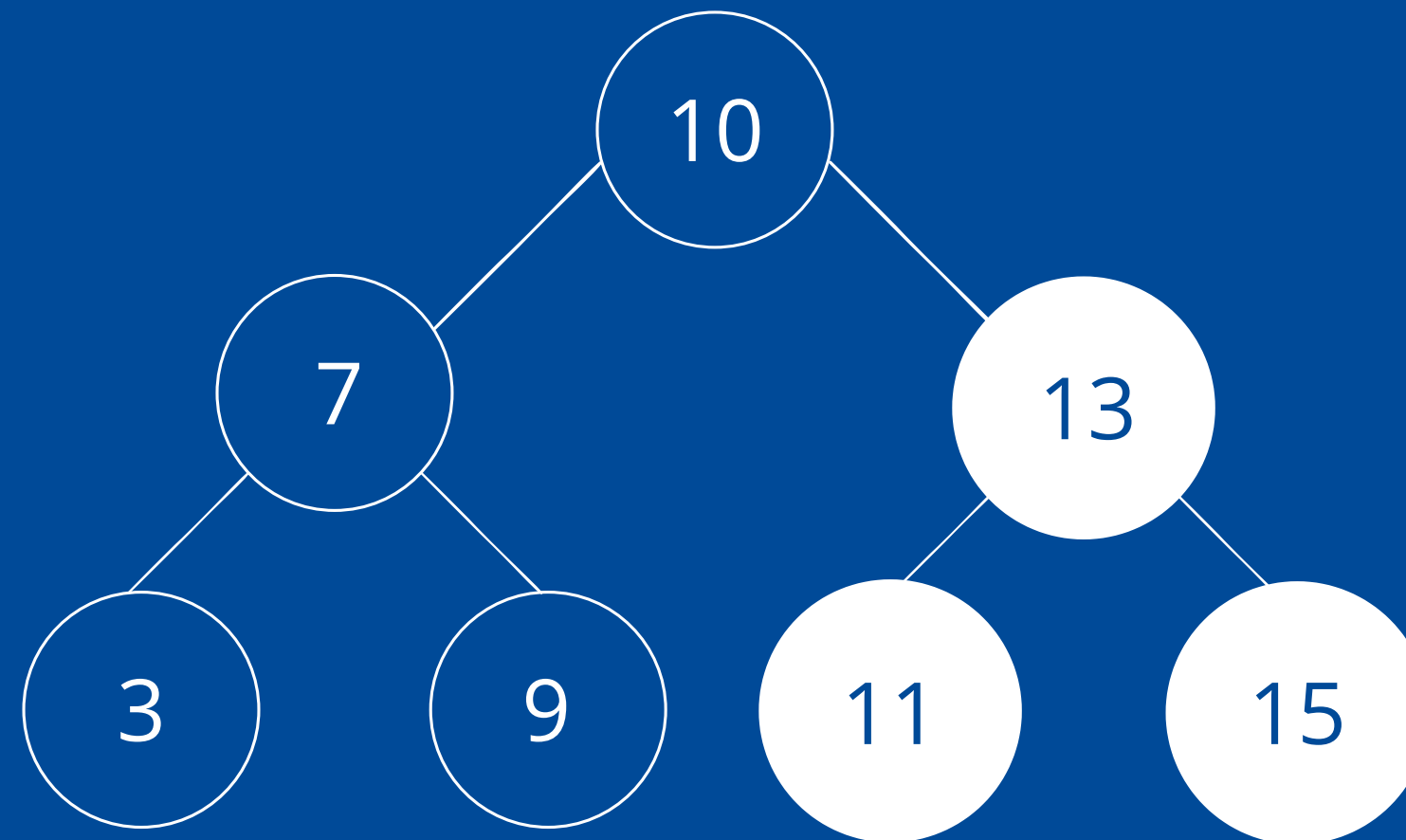
$R \leftarrow \text{Min}(D.\text{right})$

$t \leftarrow R.\text{right}$

$D.\text{key} \leftarrow R.\text{key}$

$D.\text{value} \leftarrow R.\text{value}$

$R \leftarrow t$



Entonces,

$R = D.\text{right}$

$t = \emptyset$

$D.\text{key} \leftarrow 15$

$D.\text{value} \leftarrow v$

$R = \emptyset$

# Delete en ABB

Digamos que deseo eliminar el nodo con key 13

Delete(A, 13)

else:

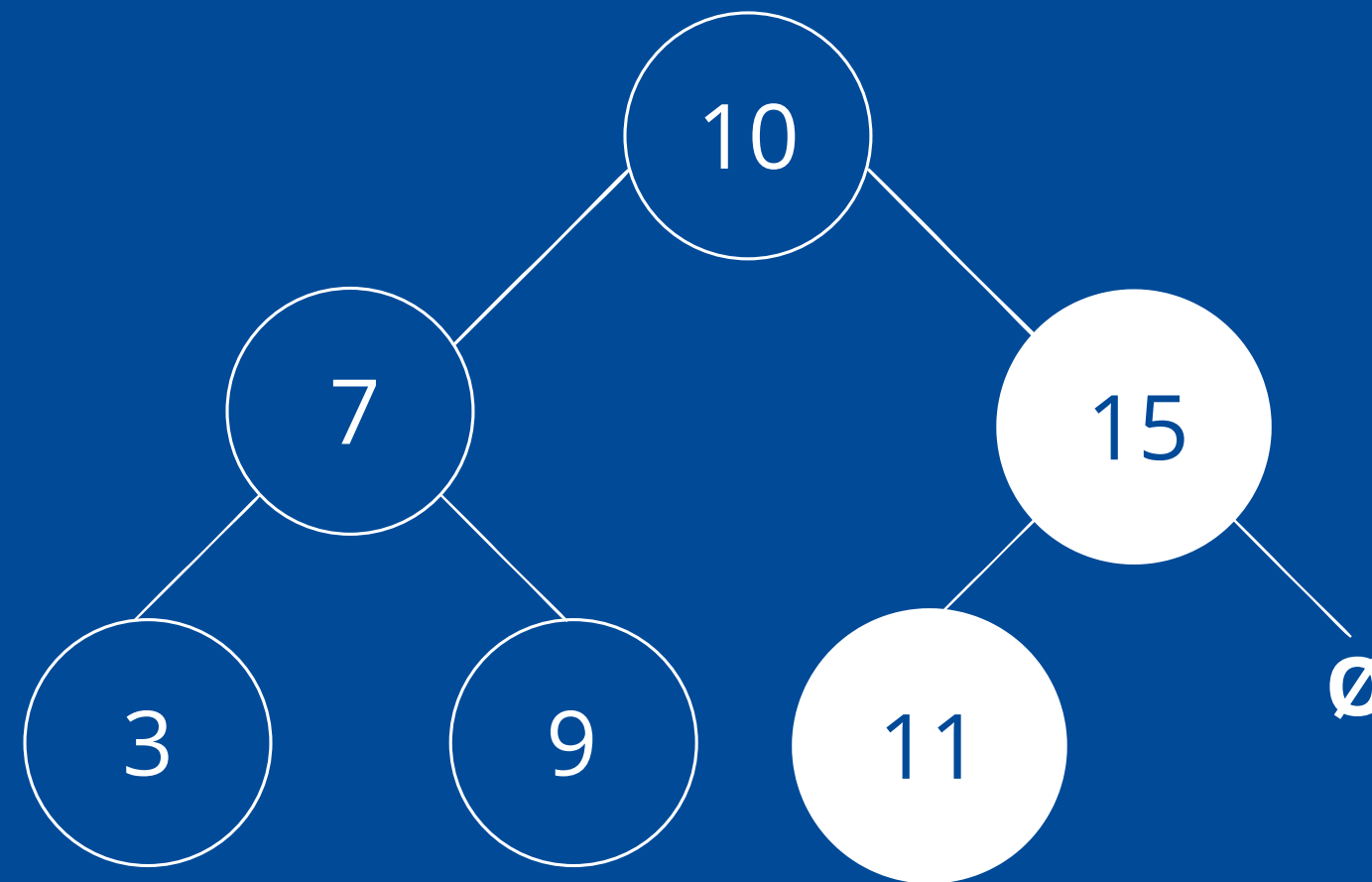
$R \leftarrow \text{Min}(D.\text{right})$

$t \leftarrow R.\text{right}$

$D.\text{key} \leftarrow R.\text{key}$

$D.\text{value} \leftarrow R.\text{value}$

$R \leftarrow t$



Cambiamos en  
el árbol:

$R = D.\text{right}$

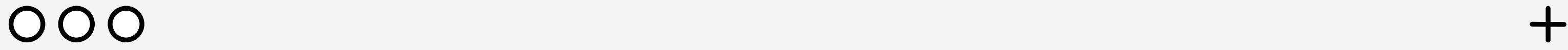
$t = \emptyset$

$D.\text{key} \leftarrow 15$

$D.\text{value} \leftarrow v$

$R = \emptyset$

# Ejercicio 1



Una empresa de logística registra cronológicamente en un archivo digital cada entrega realizada por su flota de repartidores por cuenta de sus clientes. Cada registro contiene: fecha\_entrega, hora\_entrega, rut\_cliente, id\_repartidor, id\_destino, duración\_entrega. Se desea procesar el registro señalado para enviar a cada cliente el listado con los destinos y duraciones de las entregas realizadas por cada repartidor, manteniendo el orden cronológico en que cada repartidor efectuó sus entregas.

**Pregunta 2 - Examen 2026-01**





- a) [2,0 puntos] Proponga una solución (algoritmo) para obtener la información indicada de la manera más eficiente posible en tiempo.

**Pregunta 1 - Examen 2026-01**



## Ejercicio 1a - Solución



Palabras clave en el enunciado:

debe procesar el registro señalado para enviar a cada cliente el listado con los destinos y duraciones de las entregas realizadas por cada repartidor, **manteniendo el orden cronológico** en que cada repartidor efectuó sus entregas.

- a) [2,0 puntos] Proponga una solución (algoritmo) para obtener la información indicada de **la manera más eficiente posible en tiempo.**

## Ejercicio 1a - Solución

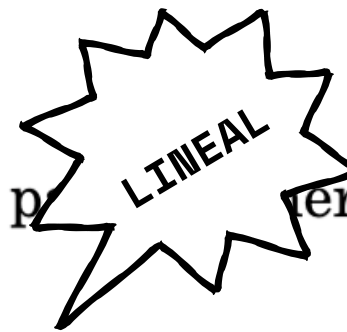


Palabras clave en el enunciado:

debe procesar el registro señalado para enviar a cada cliente el listado con los destinos y duraciones de las entregas realizadas por cada repartidor, **manteniendo el orden cronológico** en que cada repartidor efectuó sus entregas.



- a) [2,0 puntos] Proponga una solución (algoritmo) por la información indicada de **la manera más eficiente posible en tiempo.**



## Ejercicio 1a - Solución



La mejor complejidad para ordenar strings vista en el curso es  $O(n)$ .

Para esto podemos hacer uso de radixSort usando countingSort como algoritmo estable

Se debe ordenar el archivo por rut\_cliente, dado que los datos vienen originalmente ordenados cronológicamente y gracias a la estabilidad de countingSort, ya tendremos el archivo como es solicitado.



- b) [2,0 puntos] Muestre que su solución cumple los requisitos de agrupación por cliente y de preservación del orden cronológico de las entregas dentro de cada grupo.

**Pregunta 1 - Examen 2026-01**



## Ejercicio 1b - Solución



Como el archivo digital viene ordenado cronológicamente, la estabilidad entregada por countingSort al momento de ordenar por rut permitirá mantener el orden por fecha en cada subgrupo de cliente.

Así, para poder enviar a cada cliente el listado, basta recorrer revisando el rut, y al encontrar el indicado enviar los datos asociados en el orden que se encuentran.

c) [2,0 puntos] Un compañero propone aplicar *Quicksort* sobre el registro completo, ordenándolo por `rut_cliente`, y luego separar los registros para formar los grupos. Compare su propuesta con la de su compañero en términos de eficiencia y correctitud.

**Pregunta 1 - Examen 2026-01**



## Ejercicio 1c - Solución



Como QuickSort no es un algoritmo estable, luego de utilizarlo para ordenar por cliente, sería necesario reordenar cada subgrupo cronológicamente de nuevo.

La complejidad de QuickSort es  $O(n \log n)$ . Luego, si tenemos  $m$  clientes la complejidad sería de  $m * O(n \log n)$ .

Como  $O(n) < m * O(n \log n)$ , el uso de QuickSort no sería la manera más eficiente para abordar este problema.

# Ejercicio 2

3. a) Considera la siguiente afirmación respecto a un algoritmo de ordenación de strings del mismo largo:

"Otra forma de establecer la demostración de que el algoritmo es correcto es pensar en el futuro: si los caracteres que aún no han sido examinados para un par de claves son idénticos, entonces cualquier diferencia entre las claves está restringida a los caracteres que ya han sido examinados, de modo que las claves han sido ordenadas apropiadamente y permanecerán así debido a la estabilidad. Si, por el contrario, los caracteres que no han sido examinados son diferentes, entonces los caracteres ya examinados no importan y una pasada posterior ordenará correctamente el par de claves basado en esas diferencias más significativas."

¿Exactamente a cuál algoritmo para ordenación de strings se refiere esta afirmación? **Justifica.**

**Pregunta 3 - I2 2015-01**



## Ejercicio 2a - Solución



Esta afirmación está haciendo referencia a radixSort, que ordena los strings empezando por el primer caracter menos significativo (el de más a la derecha), luego según el siguiente menos significativo y así sucesivamente.

La ordenación de los strings segun un caracter determinado debe hacerse usando un algoritmo de ordenación estable, como lo es countingSort.



b) Otra forma de ordenar strings, especialmente cuando son de diferentes largos, es considerar los caracteres de izquierda a derecha y usar el siguiente método recursivo: Usamos *countingSort()* para ordenar los strings de acuerdo con el primer carácter; luego, recursivamente, ordenamos los strings que tienen un mismo primer carácter (excluyendo este primer carácter). Similarmente a *quickSort()*, este algoritmo particiona el arreglo de strings en subarreglos que pueden ser ordenados independientemente para completar la tarea, sólo que particiona el arreglo en un subarreglo para cada posible valor del primer carácter, en lugar de las dos particiones de *quickSort()*. **Escribe este algoritmo.**

**Pregunta 3 - I2 2015-01**



## Ejercicio 2b - Solución



Sea el algoritmo `sortString(a, e, w k)` nuestra solución, que:

- ordena recursivamente el arreglo de strings `a`, desde `a[e]` hasta `a[w]`, a partir del **k-ésimo** carácter
- convierte cada carácter a un dígito entre 0 y 127 mediante la función (ficticia) `dgt`

## Ejercicio 2b - Solución



```
sortString(a, e, w, k):
    if e < w:
        for i = e, ..., w:
            p = dgt(a[i][k])
            b[p+2] = b[p+2]+1
        for r = 0, ..., 128:
            b[r+1] = b[r+1] + b[r]
        for i = e, ..., w:
            p = dgt(a[i][k])
            c[b[p+1]] = a[i]
            b[p+1] = b[p+1]+1
        for i = e, ..., w:
            a[i] = c[i-e]
        for r = 0, ..., 127:
            sortString(a, e+b[r], e+b[r+1]-1, k+1)
```

- Sean b y c arreglos auxiliares
- Los primeros 4 for implementan countingSort
- El quinto for hace las llamadas recursivas sobre cada una de las particiones (grupos de strings con el mismo k-ésimo caracter)

# Ejercicio 3

(2 pts) El Profesor Eterovic indica que ha descubierto una propiedad notable de los ABB. Suponga que la búsqueda de la clave  $k$  en un ABB termina en una hoja. Consideremos tres conjuntos:  $A$ , las claves a la izquierda de la ruta de búsqueda;  $B$ , las claves en la ruta de búsqueda; y  $C$ , las claves a la derecha de la ruta de búsqueda. El profesor Eterovic afirma que tres claves cualesquiera  $a \in A, b \in B$  y  $c \in C$  deben satisfacer  $a \leq b \leq c$ . ¿Será cierto? En caso afirmativo, demuestre formalmente esta propiedad; en caso negativo, muestre con un ejemplo que la propiedad no se cumple.

**Pregunta 2 - I1 2025-01**

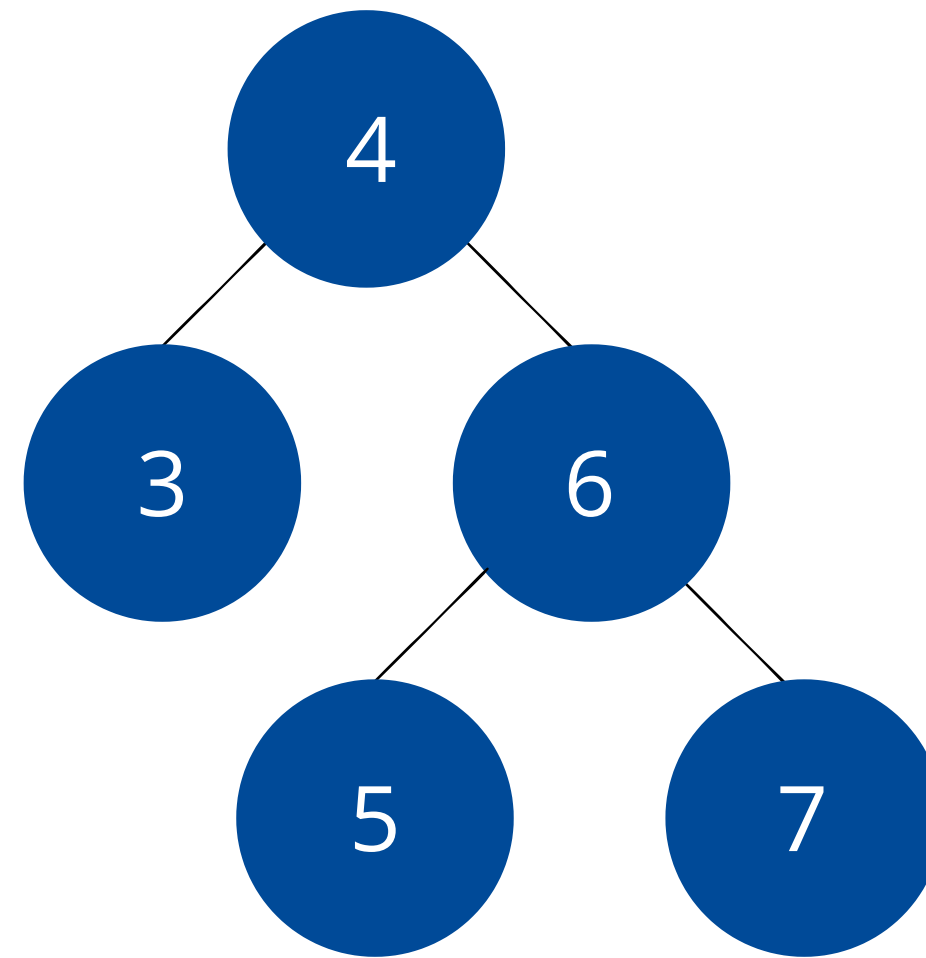


## Ejercicio 3 - Solución

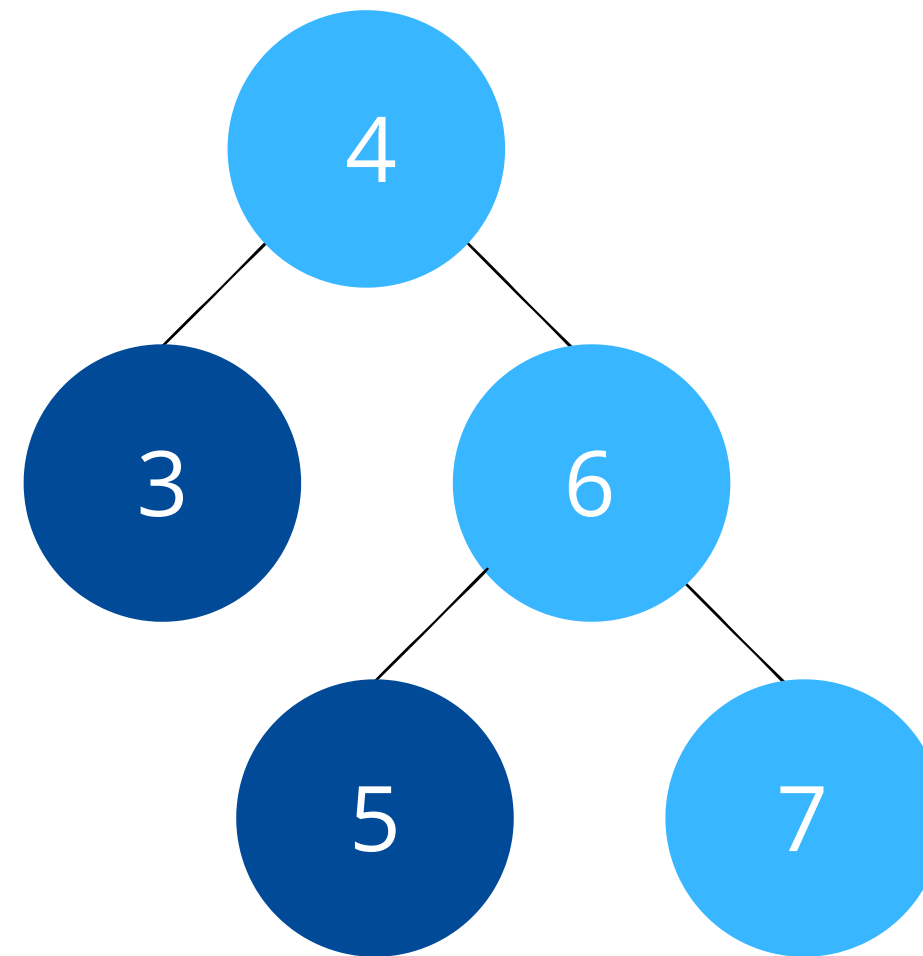


La propiedad **no es cierta**, por lo que basta mostrar un contraejemplo:  
considera el ABB con raíz 4 e hijos 3 y 6, en que 6 tiene a su vez hijos 5 y 7. Considera la búsqueda del 7: ruta 4, 6, 7  $\Rightarrow$  a la izquierda quedan las claves 3 y 5, en que  $5 > 4$

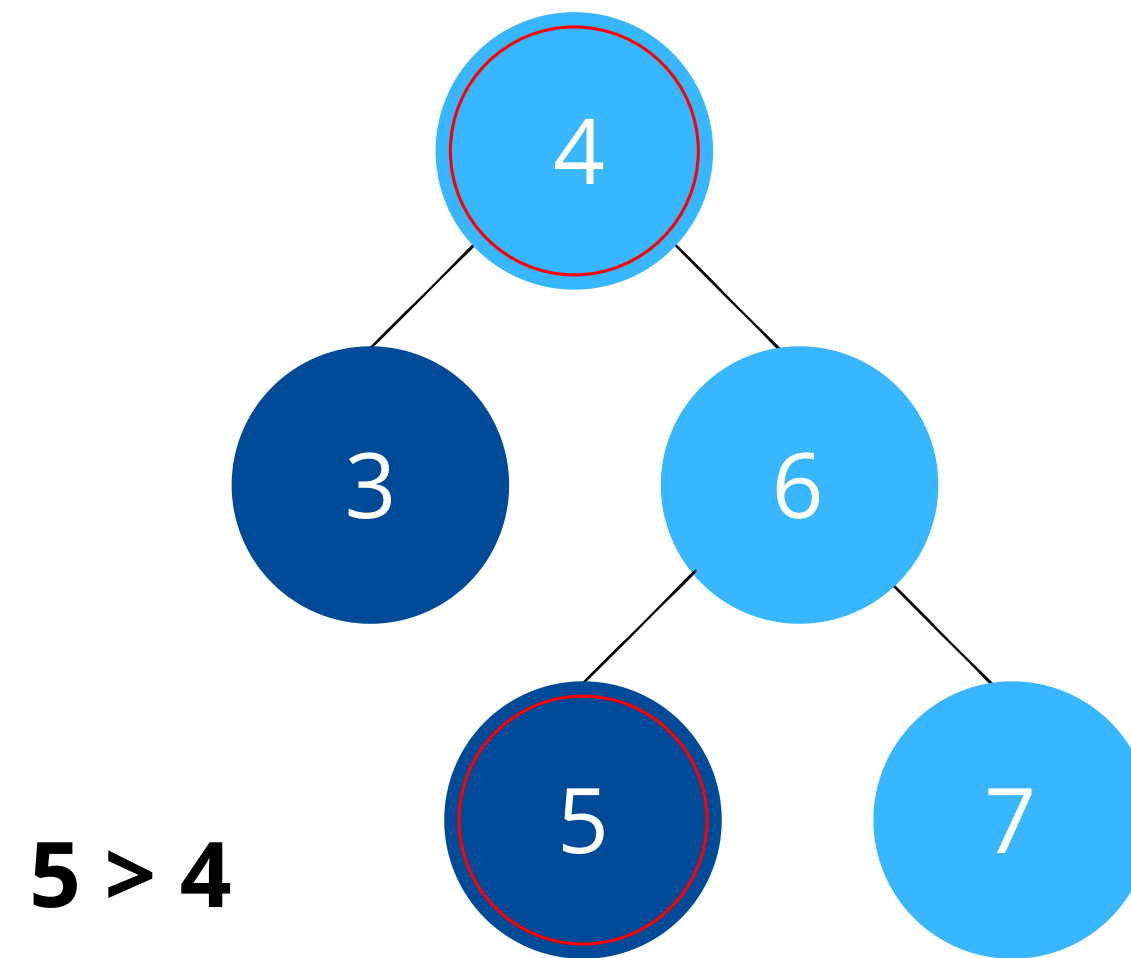
## Ejercicio 3 - Solución



## Ejercicio 3 - Solución



## Ejercicio 3 - Solución



# Ejercicio 4



#### 4) Árboles binarios de búsqueda

- a) Dado un **ABB** (no necesariamente balanceado), una forma —distinta de la estudiada en clases— de eliminar una clave cuyo nodo tiene ambos hijos es la siguiente: el nodo es ocupado por su (hijo y) subárbol izquierdo,  $T_i$ , mientras que su subárbol derecho,  $T_d$ , se convierte en el subárbol derecho del nodo más a la derecha de  $T_i$ .
- i) [1 pt.] Justifica que esta eliminación respeta las propiedades de ABB.
- ii) [0.5 pts.] Muestra con ejemplos que esta eliminación puede tanto aumentar como reducir la altura del árbol original (dos ejemplos, uno para cada caso).

**Pregunta 4 - I1-2025-2**



## Ejercicio 4 - Solución



a) i) A partir de la regla para insertar claves, que, a su vez, cumple la propiedad fundamental de ABB, sabemos que en el proceso de inserción de cualquiera de las claves que están en  $T_i$  o  $T_d$  —llamemos  $k$  a una clave cualquiera en  $T_i$  o  $T_d$ — pasamos por la clave —llamémosla  $j$ — que estamos eliminando, y que, por lo tanto,  $k$  podría haber ido a parar al lugar de  $j$ , posición en la que habría cumplido la propiedad de ABB con respecto al resto del árbol. Por lo tanto, poner el subárbol  $T_i$  en la posición que ocupaba el nodo con clave  $j$  es válido.

La pregunta entonces es, ¿qué hacemos con  $T_d$ ? De nuevo, por la propiedad de ABB, las claves de  $T_d$  son todas mayores que las claves de  $T_i$ . La única posición que corresponde a claves mayores que todas las claves de  $T_i$ , pero al mismo tiempo menores que las otras claves del árbol mayores que  $j$  es como hijo derecho de la clave más a la derecha de  $T_i$  —llamémosla  $m$ ; obviamente, esta posición está "desocupada":  $m$  sólo puede ser la clave más a la derecha de  $T_i$  si no tiene hijo derecho.

## Ejercicio 4 - Solución



a) ii) Para simplificar (y generalizar un poco), eliminamos la raíz del árbol. Entonces,  $T_i$  "sube" a esta posición y agregamos  $T_d$  como hijo derecho del nodo más a la derecha de  $T_i$ .

La altura del árbol original,  $T$ , era  $H(T) = \max\{H(T_i), H(T_d)\} + 1$ . La altura del nuevo árbol,  $T'$ , puede ser desde  $H(T') = H(T_i)$  hasta  $H(T') = H(T_i) + H(T_d)$ , dependiendo de la profundidad del nodo más a la derecha de  $T_i$ . En el primer caso,  $H(T')$  es claramente menor que  $H(T)$ ; en el segundo,  $H(T')$  claramente puede ser mayor que  $H(T)$ .

# Feedback

